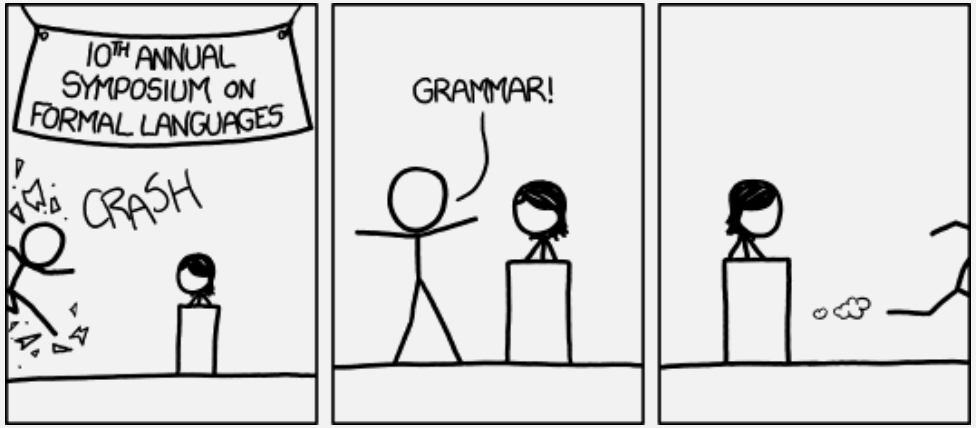
CS 420

Pushdown Automata (PDAs)

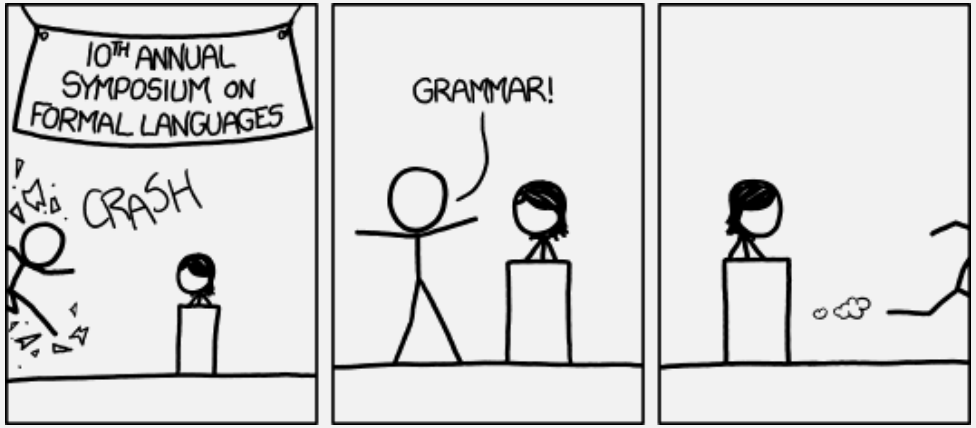
Tuesday October 27, 2026

UMass Boston Computer Science



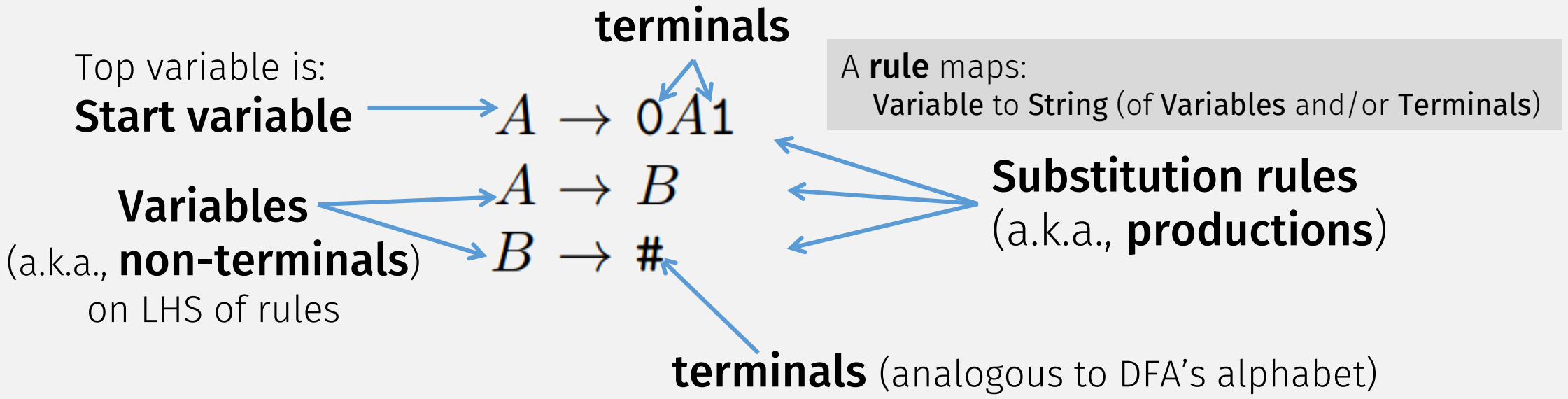
Announcements

- HW 6
  - Due: Thur 10/29 12:30pm



Last Time:

A Context-Free Grammar (CFG)



Last Time:

Context-Free Grammar (CFG): Formally

let G_1 = **mk-CFG** (V, Σ, R, S) …

R is this **set of rules: pair of Var, String** (of vars+terminals)

Top variable is:
Start variable → $A \to 0A1$

Variables (a.k.a., **non-terminals**) → $A \to B$, $B \to \#$

Substitution rules (a.k.a., **productions**)

terminals (analogous to DFA's alphabet)

A ***context-free grammar*** is a `mk-CFG` (V, Σ, R, S), where

1. V is a finite set called the ***variables***,
2. Σ is a finite set, disjoint from V, called the ***terminals***,
3. R is a finite set of ***rules***, with each rule being a variable and a string of variables and terminals, and
4. $S \in V$ is the start variable.

where:

$V = \{A, B\}$,

$\Sigma = \{0, 1, \#\}$,

$S = A$,

Last Time:

Definition:
A **CFG** describes a **context-free language!**

DefCFL

(some defined CFG)

IF G IS-A **Context-free** Grammar
AND
G **describes** L (some defined language)
THEN L IS-A **Context-free** Language

(abbrev)

DefCFL

IF G IS-A **CFG**
AND
G **describes** L
THEN L IS-A **CFL**

Generating Strings with a CFG

Called "context free" because: we replace vars with RHS of rule, without looking at surrounding chars, i.e., context

Definition:
A **CFG** describes a **context-free language!**

but what strings are in the language?

1st rule → $A \to 0A1$

2nd rule → $A \to B$

Last rule → $B \to \#$

"Applying a rule"
= replace LHS variable with RHS sequence

At each step, *arbitrarily* choose any variable to replace, and any rule to apply

Stop when: string is all terminals

A CFG **generates** a string, by repeatedly applying substitution rules:

Example: $A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$

Start with: Start variable

Apply 1st rule

1st rule again

1st rule again

Apply 2nd rule

Apply last rule

Generating Strings with a CFG

let $G_1 =$

$$A \to 0A1$$
$$A \to B$$
$$B \to \#$$

Strings in CFG's language
= all possible **generated** / **derived** strings

$$\textsc{LangOf}(G_1) = \{0^n\#1^n \mid n \geq 0\}$$

A CFG **generates** a string, by repeatedly applying substitution rules:

Example: $A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$

This sequence of steps is called a **derivation**

Derivations: Formally

A ***context-free grammar*** is a `mk-CFG` (V, Σ, R, S), where

1. V is a finite set called the ***variables***,
2. Σ is a finite set, disjoint from V, called the ***terminals***,
3. R is a finite set of ***rules***, with each rule being a variable and a string of variables and terminals, and
4. $S \in V$ is the start variable.

let G = **`mk-CFG`** (V, Σ, R, S) ...

Single-step

"context" stays same

$$\alpha A \beta \underset{G}{\Rightarrow} \alpha \gamma \beta$$

Where:

sequence of: variables or terminals

$\gamma, \alpha, \beta \in (V \cup \Sigma)^*$

$A \in V$ ← Variable

$A \to \gamma \in R$ ← Rule

Derivations: Formally

A ***context-free grammar*** is a `mk-CFG` (V, Σ, R, S), where

1. V is a finite set called the ***variables***,
2. Σ is a finite set, disjoint from V, called the ***terminals***,
3. R is a finite set of ***rules***, with each rule being a variable and a string of variables and terminals, and
4. $S \in V$ is the start variable.

let G = **`mk-CFG`** (V, Σ, R, S) ...

Single-step

$$\alpha A \beta \underset{G}{\Rightarrow} \alpha \gamma \beta$$

Where:

$$\alpha, \beta \in (V \cup \Sigma)^*$$

$$A \in V \leftarrow \text{Variable}$$

$$A \to \gamma \in R \leftarrow \text{Rule}$$

Multi-step (recursively defined, on # steps)

Base case: $\alpha \overset{*}{\underset{G}{\Rightarrow}} \alpha$ (0 steps)

Recursive case: (1 or more steps)

Single step

- If: $\alpha \underset{G}{\Rightarrow} \beta$ and $\beta \overset{*}{\underset{G}{\Rightarrow}} \gamma$
- Then: $\alpha \overset{*}{\underset{G}{\Rightarrow}} \gamma$

(smaller) Recursive "call"

Lang of a CFG, Formally

A ***context-free grammar*** is a `mk-CFG` (V, Σ, R, S), where

1. V is a finite set called the ***variables***,
2. Σ is a finite set, disjoint from V, called the ***terminals***,
3. R is a finite set of ***rules***, with each rule being a variable and a string of variables and terminals, and
4. $S \in V$ is the start variable.

let G = **`mk-CFG`** (V, Σ, R, S) ...

"the language of a grammar G is ..."

"all possible sequences of terminal symbols ..."

... "that can be **generated** with **rules of grammar** G"

$$\textsc{LangOf}(G) = \left\{ w \in \Sigma^* \mid S \overset{*}{\underset{G}{\Rightarrow}} w \right\}$$

Designing Grammars : Basics

1. Think about what you want to “link” together

Last Time:

Non-Regular Languages

Example:

An arbitrary count

$$L = \{ \mathtt{0}^n\mathtt{1}^n \mid n \geq 0 \}$$

- A DFA recognizing L would **require infinite states!** (impossible)
  - States representing: **zero `0`s seen, one `0` seen, two `0`s, …**
- **This language is the same as many PLs,** e.g., **HTML!**
  - To better see this replace:
    - "`0`" with "`<tag>`" or "`(`"
    - "`1`" with "`</tag>`" or "`)`"
- The Problem: **remembering <u>nestedness</u>**
  - **Need to count arbitrary nesting depths**
    - E.g., `{ { { … } } }`
  - Thus: **most programming language syntax** is **<u>not regular</u>!**

A regular lang can't "link" together open/close

Designing Grammars : Basics

1. **Think about what you want to "link" together**

- E.g., 0^n1^n
  - $A \rightarrow 0A1$
  - # 0s and # 1s are **"linked"**

- E.g., **HTML**
  - $\text{ELEMENT} \rightarrow \text{<TAG>CONTENT</TAG>}$
  - **Start** and **end tags** are **"linked"**

2. **Start** with **small grammars** (computation) and **then combine**
   - just like with **DFA**s, **NFA**s, and **programming**!

In-class exercise: Creating CFG

alphabet Σ is $\{0,1\}$

$$L = \{w \mid w \text{ starts and ends with the same symbol}\}$$

1) come up with examples: In the language: **010, 101, 110001** **1, 0 ?** ☑

Not in the language: **10, 01, 110** **ε ?** ☒

2) Create CFG:

Needed Rules:

$S \rightarrow 0M0 \mid 1M1 \mid 0 \mid 1$ "start/end symbol are "linked" (ie, same); middle can be anything"

$M \rightarrow MT \mid \varepsilon$ "middle: all possible terminals, repeated (ie, all possible strings)"

$T \rightarrow 0 \mid 1$ "all possible terminals"

3) Check CFG: generates examples in the language; doesn't generate examples not in language

Examples Table! (justifies that the "CFG describes L")

Designing Grammars: Building Up

- **Start** with **small grammars** and then **combine** (just like **programming**)
  - To create a **grammar for the language** $\{0^n 1^n | n \geq 0\} \cup \{1^n 0^n | n \geq 0\}$
  - First create **grammar for lang** $\{0^n 1^n | n \geq 0\}$:

$$S_1 \rightarrow 0S_1 1 \mid \varepsilon$$

  - Then create **grammar for lang** $\{1^n 0^n | n \geq 0\}$:

$$S_2 \rightarrow 1S_2 0 \mid \varepsilon$$

  - Then **combine**:

$$S \rightarrow S_1 \mid S_2$$
$$S_1 \rightarrow 0S_1 1 \mid \varepsilon$$
$$S_2 \rightarrow 1S_2 0 \mid \varepsilon$$

New start variable and **rule combines two smaller grammars**

"|" = "or" = union (combines 2 rules with same left side)

Last Time:

(Closed) Operations for CFLs?

- **Start** with **small grammars** and then **combine** (just like **programming**)
- "Or": $S \to S_1 \mid S_2$
- "Concatenate": $S \to S_1 S_2$
- "Star" (repetition): $S' \to S' S_1 \mid \varepsilon$

Status check:
Could you write out the precise **Statement to Prove** and the **full proof?**

"The set of CFLs are closed under …"

"IF L_1 IS-A CFL AND L_2 IS-A CFL THEN … IS-A CFL"

Regular Language vs CFL Comparison

Regular Languages	Context-Free Languages (CFLs)
Regular Expression	Context-Free Grammar (CFG)
describes a **Regular Lang**	describes a **CFL**

Regular Language vs CFL Comparison

Regular Languages	Context-Free Languages (CFLs)
Regular Expression	Context-Free Grammar (CFG)
describes a **Regular Lang**	describes a **CFL**
Finite State Automaton (FSM)	???
recognizes a **Regular Lang**	recognizes a **CFL**

Regular Language vs CFL Comparison

Regular Languages	Context-Free Languages (CFLs)
Regular Expression	Context-Free Grammar (CFG)
describes a **Regular Lang**	describes a **CFL**
Finite State Automaton (FSM)	**Push-down Automata** (PDA)
recognizes a **Regular Lang**	recognizes a **CFL**

thm

def

def

thm

Regular Language vs CFL Comparison

Regular Languages	Context-Free Languages (CFLs)
Regular Expression	Context-Free Grammar (CFG)
describes a **Regular Lang**	describes a **CFL**
Finite State Automaton (FSM)	**Push-down Automata** (PDA)
recognizes a **Regular Lang**	recognizes a **CFL**
Proved:	Must Prove:
Regular Lang ⇔ Regular Expr ☑	CFL ⇔ PDA ???

thm

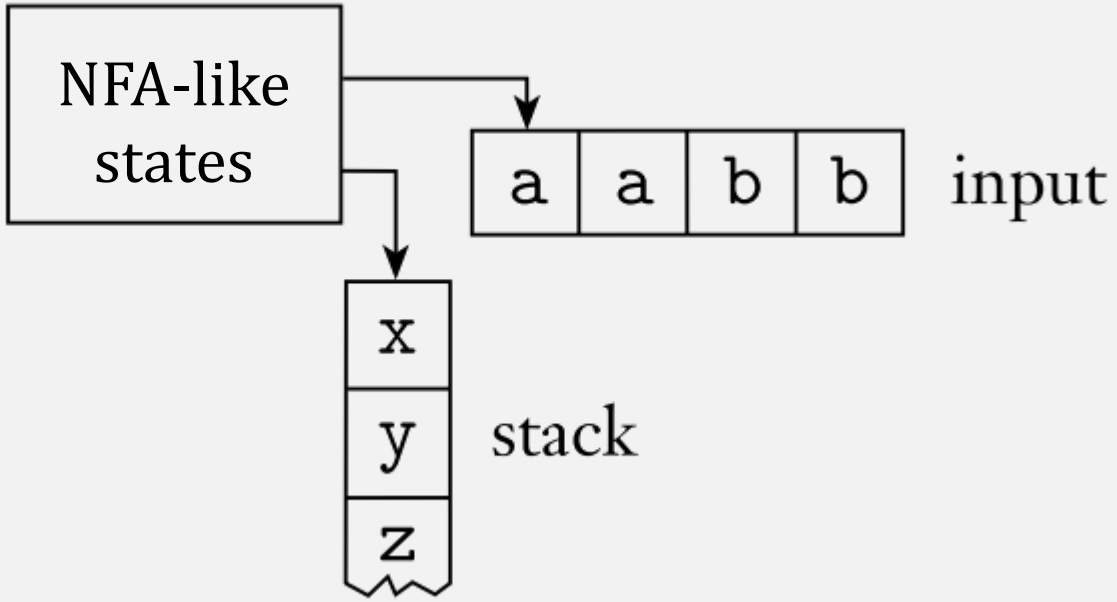
def

def

thm

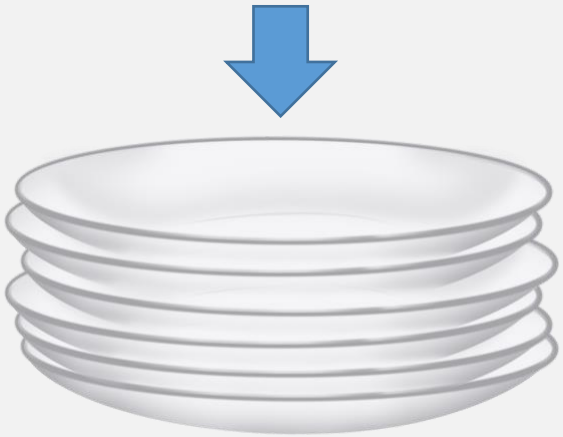
Pushdown Automata (PDA)

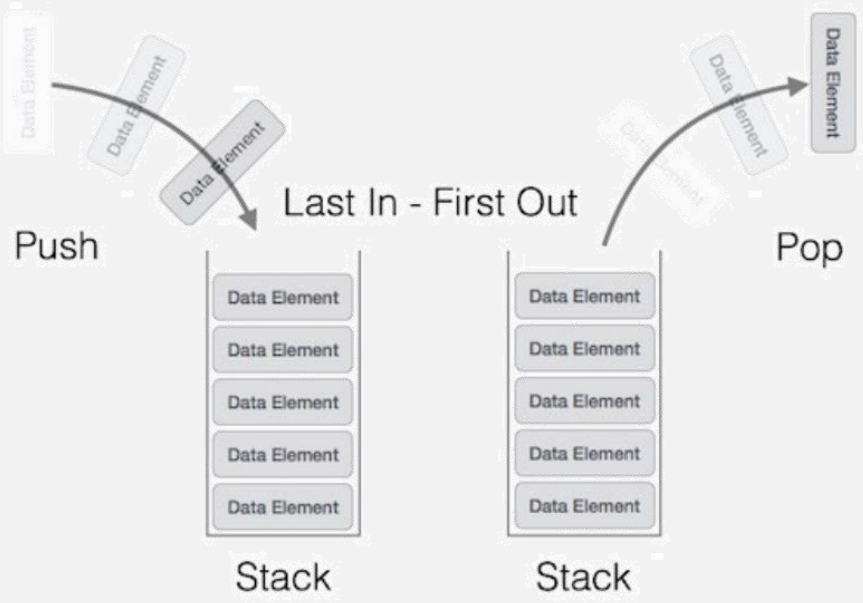
PDA = NFA + a stack



What is a Stack?

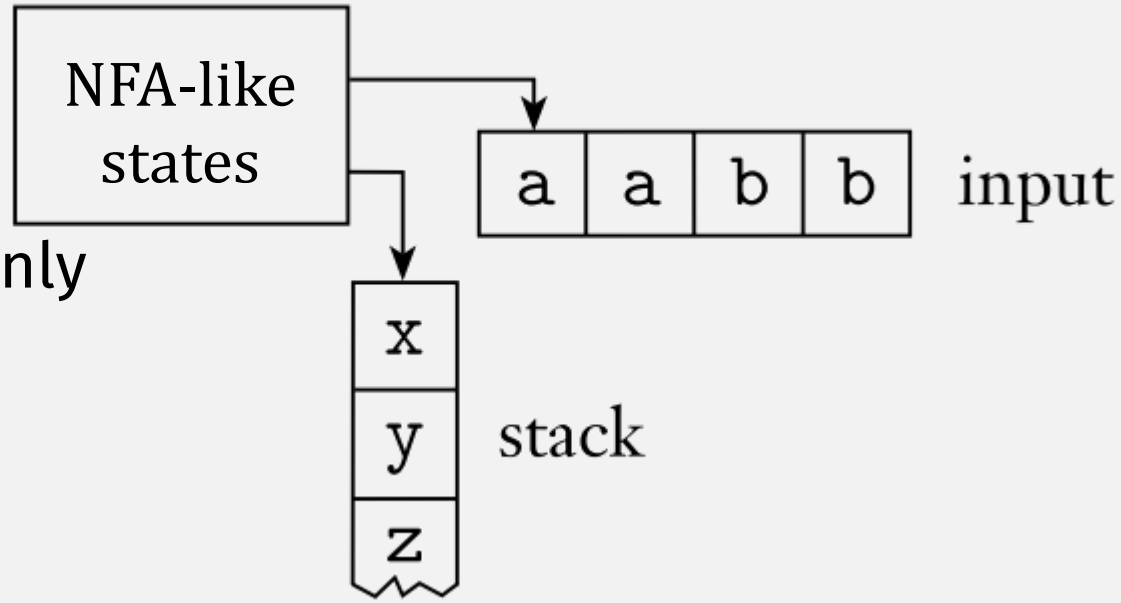
- A **restricted** kind of (**infinite!**) **memory**
- **Access** to **top element only**
- 2 Operations only: **push, pop**





Pushdown Automata (PDA)

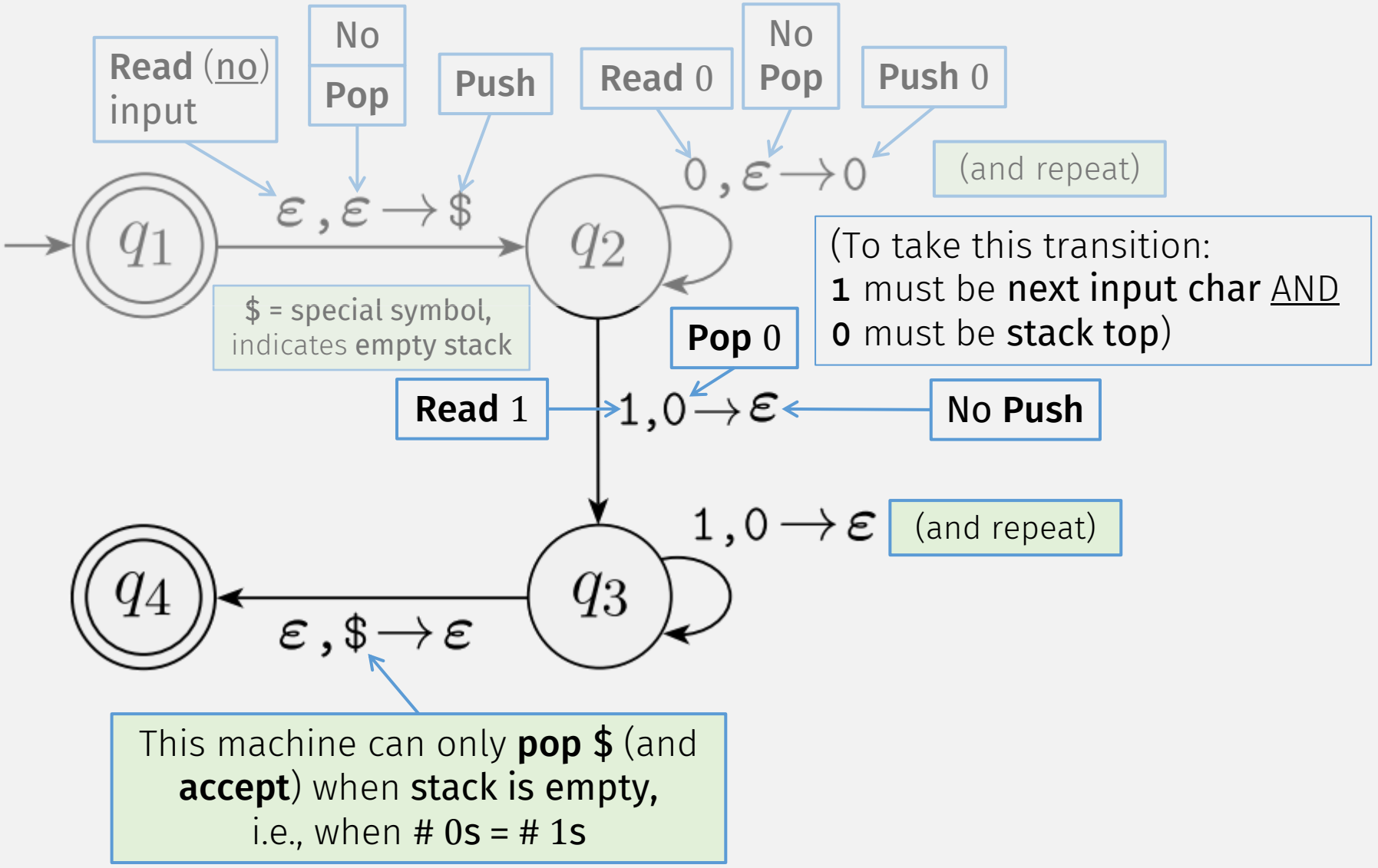
- **PDA = NFA + a stack**
  - Infinite memory!
  - But … read/write top location only
    - Push/pop



$$\{0^n 1^n \mid n \geq 0\}$$

An Example PDA

A **PDA transition** has <u>3 parts:</u>
- **Read** (input)
- **Pop** (stack)
- **Push** (stack)



Formal Definition of PDA

A ***pushdown automaton*** is a `mk-PDA` $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q, Σ, Γ, and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Stack alphabet has special stack symbols, e.g., **$**

Input

Pop

Push

Non-deterministic!
Result of a step is **set** of (STATE, STACK CHAR) pairs

let M_1 =
mk-PDA (Q, Σ, Γ, δ, q_1, F)

PDA Formal Definition Example

where …

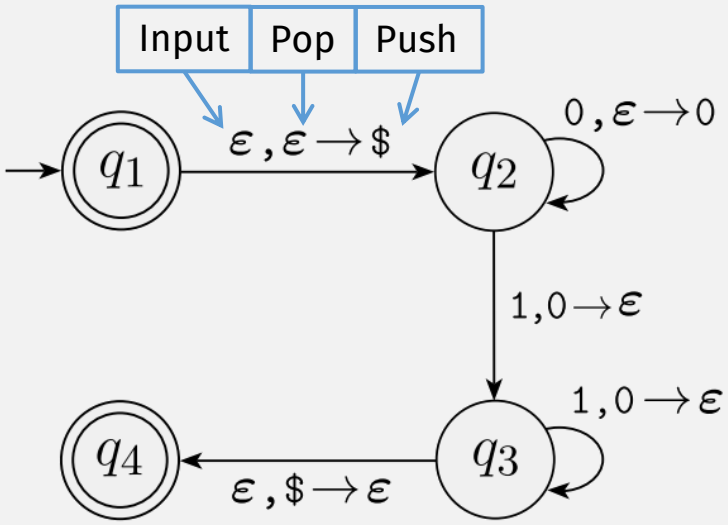
$$Q = \{q_1, q_2, q_3, q_4\},$$

$$\Sigma = \{0, 1\},$$

$$\Gamma = \{0, \$\},$$

$$F = \{q_1, q_4\},$$

Stack alphabet has special stack symbol $



A ***pushdown automaton*** is a `mk-PDA` $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q, Σ, Γ, and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Input
Pop
Push

let M_1 =
mk-PDA (Q, Σ, Γ, δ, q_1, F)
where ...

$$Q = \{q_1, q_2, q_3, q_4\},$$

$$\Sigma = \{0,1\},$$

$$\Gamma = \{0, \$\},$$

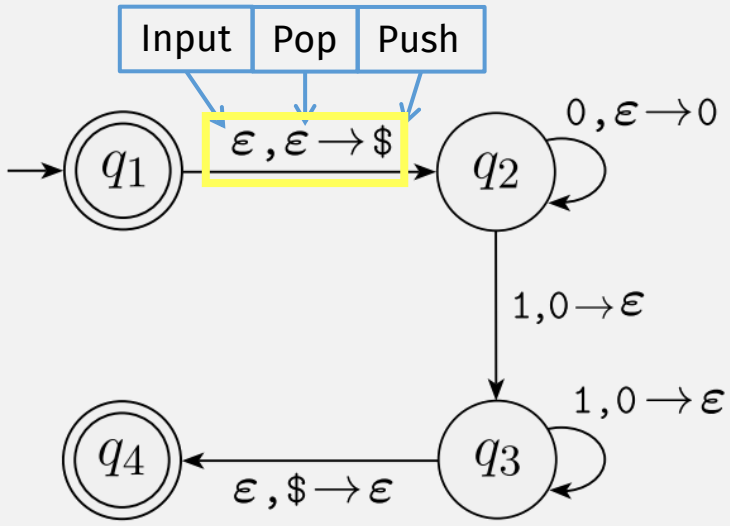
$$F = \{q_1, q_4\}, \text{ and}$$

δ is given by the following table, wherein blank entries signify $\emptyset$.

Input:	0			1			ε		
Stack:	0	\$	ε	0	\$	ε	0	\$	ε
q_1									$\{(q_2, \$)\}$ 5
q_2			$\{(q_2, 0)\}$ 1		$\{(q_3, \varepsilon)\}$ 2			$\{(q_3, \varepsilon)\}$ 4	
q_3					$\{(q_3, \varepsilon)\}$ 3				$\{(q_4, \varepsilon)\}$
q_4									

Input
Pop
Push

Let's play a game:
"Match transition to a number!"



A ***pushdown automaton*** is a **mk-PDA** $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q, Σ, Γ, and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Input
Pop
Push

let M_1 =
mk-PDA (Q, Σ, Γ, δ, q_1, F)
where ...

$$Q = \{q_1, q_2, q_3, q_4\},$$

$$\Sigma = \{0,1\},$$

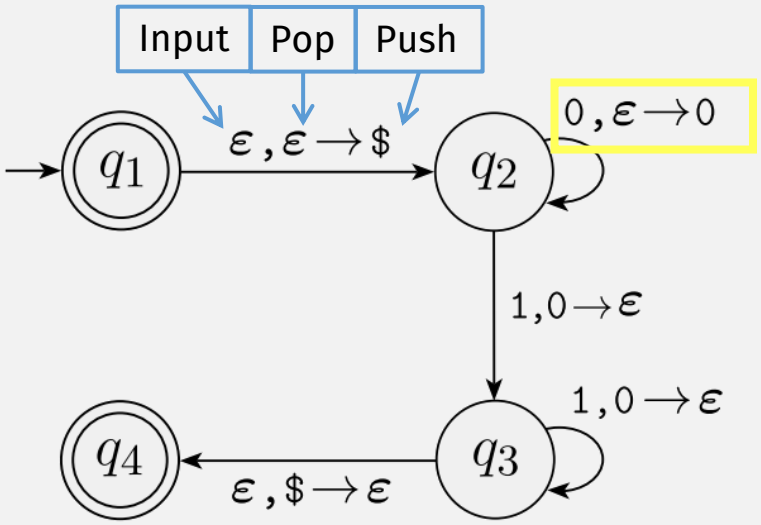
$$\Gamma = \{0, \$\},$$

$$F = \{q_1, q_4\}, \text{ and}$$

δ is given by the following table, wherein blank entries signify $\emptyset$.

Input:	0			1			ε		
Stack:	0	\$	ε	0	\$	ε	0	\$	ε
q_1									$\{(q_2, \$)\}$ 5
q_2			$\{(q_2, 0)\}$ 1		$\{(q_3, \varepsilon)\}$ 2			$\{(q_3, \varepsilon)\}$ 4	
q_3					$\{(q_3, \varepsilon)\}$ 3			$\{(q_4, \varepsilon)\}$	
q_4									

Input
Pop
Push



A ***pushdown automaton*** is a `mk-PDA` $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q, Σ, Γ, and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Input
Pop
Push

let M_1 =
mk-PDA (Q, Σ, Γ, δ, q_1, F)
where ...

$$Q = \{q_1, q_2, q_3, q_4\},$$

$$\Sigma = \{0,1\},$$

$$\Gamma = \{0, \$\},$$

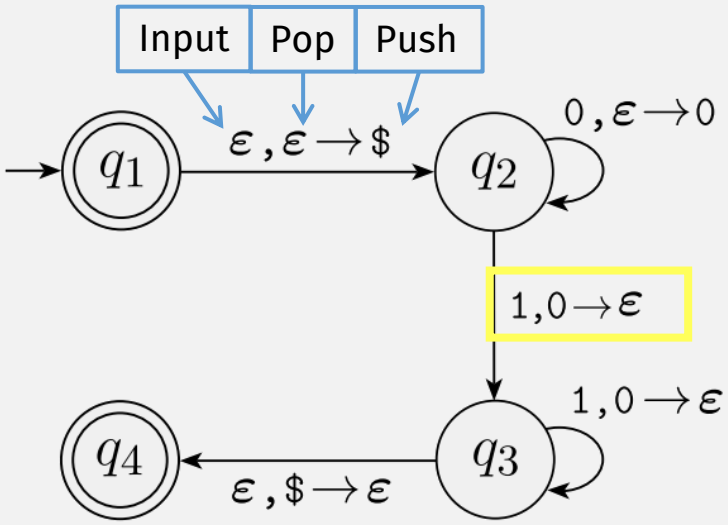
$$F = \{q_1, q_4\}, \text{ and}$$

δ is given by the following table, wherein blank entries signify $\emptyset$.

Input:	0			1			ε		
Stack:	0	\$	ε	0	\$	ε	0	\$	ε
q_1									$\{(q_2, \$)\}$
q_2		$\{(q_2, 0)\}$		$\{(q_3, \varepsilon)\}$					
q_3				$\{(q_3, \varepsilon)\}$			$\{(q_4, \varepsilon)\}$		
q_4									

Input
Pop
Push

1 2 3 4 5



A ***pushdown automaton*** is a mk-PDA $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q, Σ, Γ, and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Input
Pop
Push

let M_1 =
mk-PDA (Q, Σ, Γ, δ, q_1, F)
where ...

$$Q = \{q_1, q_2, q_3, q_4\},$$

$$\Sigma = \{0,1\},$$

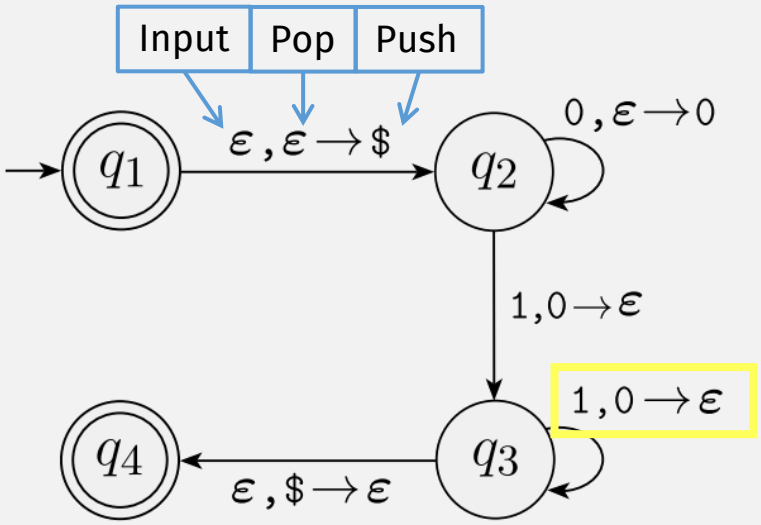
$$\Gamma = \{0, \$\},$$

$$F = \{q_1, q_4\}, \text{ and}$$

δ is given by the following table, wherein blank entries signify $\emptyset$.

Input:	0			1			ε		
Stack:	0	\$	ε	0	\$	ε	0	\$	ε
q_1									$\{(q_2, \$)\}$ 5
q_2		$\{(q_2, 0)\}$ 1		$\{(q_3, \varepsilon)\}$ 2			$\{(q_4, \varepsilon)\}$ 4		
q_3				$\{(q_3, \varepsilon)\}$ 3					
q_4									

Input
Pop
Push



A ***pushdown automaton*** is a **mk-PDA** $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q, Σ, Γ, and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Input
Pop
Push

let M_1 =
mk-PDA (Q, Σ, Γ, δ, q_1, F)
where ...

$Q = \{q_1, q_2, q_3, q_4\}$,

$\Sigma = \{0,1\}$,

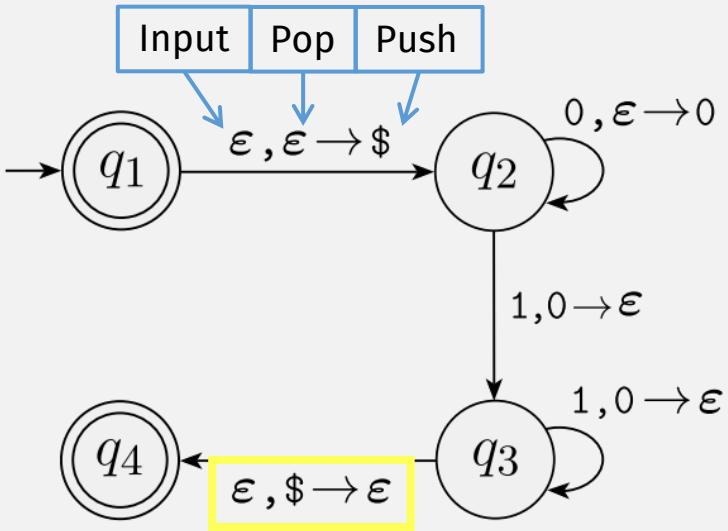
$\Gamma = \{0, \$\}$,

$F = \{q_1, q_4\}$, and

δ is given by the following table, wherein blank entries signify $\emptyset$.

Input:	0			1			ε		
Stack:	0	$	ε	0	$	ε	0	$	ε
q_1									$\{(q_2, \$)\}$ 5
q_2			$\{(q_2, 0)\}$ 1			$\{(q_3, \varepsilon)\}$ 2		$\{(q_4, \varepsilon)\}$ 4	
q_3						$\{(q_3, \varepsilon)\}$ 3			
q_4									

Input
Pop
Push



A ***pushdown automaton*** is a mk-PDA $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q, Σ, Γ, and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Input
Pop
Push

let M_3 =
mk-PDA (Q, Σ, Γ, δ, q_1, F)
where ...

$Q =$

$\Sigma =$

$\Gamma =$

$F =$

In-class exercise:
Fill in the blanks

δ is given by the following table, wherein blank entries signify $\emptyset$.

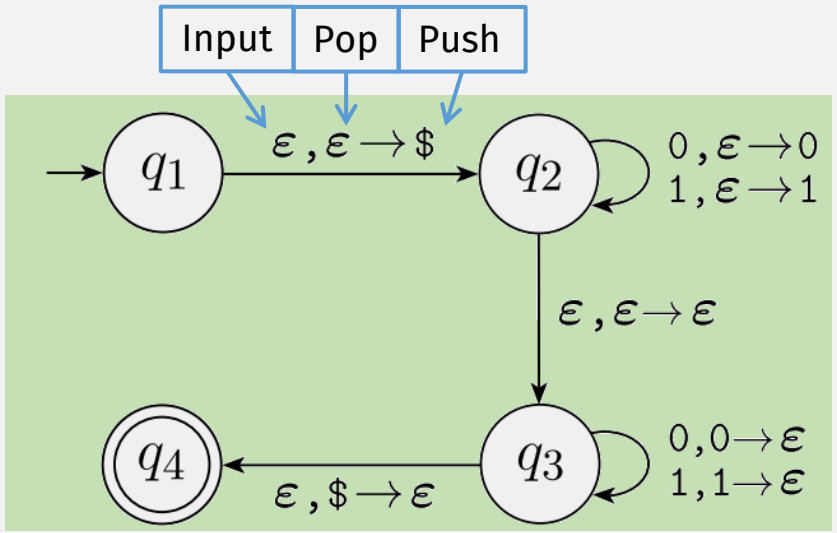
Input:	0	1	ε
Stack:	???	???	???
?			
?			

Input

Pop

State/ Push

PDA M_3 recognizing the language $\{ww^{\mathcal{R}} \mid w \in \{0,1\}^*\}$



A ***pushdown automaton*** is a mk-PDA $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q, Σ, Γ, and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ is the transition function,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Input

Pop

Push

let M_3 =
mk-PDA (Q, Σ, Γ, δ, q_1, F)
where ...

$Q = \{q_1, q_2, q_3, q_4\}$,

$\Sigma = \{0,1\}$,

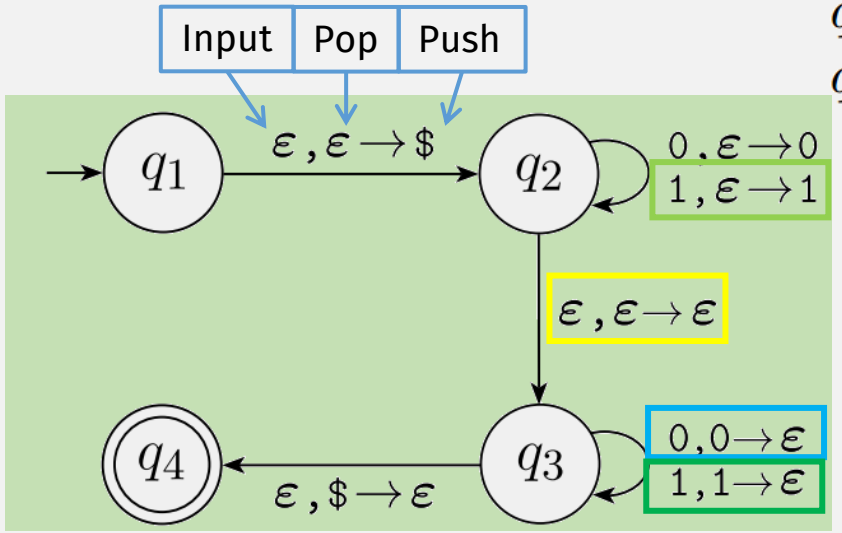
$\Gamma = \{0,1,\$\}$,

$F = \{q_4\}$

δ is given by the following table, wherein blank entries signify $\emptyset$.

Input:	0			1				ε		
Stack:	0	\$	ε	0	1	\$	ε	0	\$	ε
q_1										$\{(q_2, \$)\}$
q_2		$\{(q_2, 0)\}$				$\{(q_2, 1)\}$				$\{(q_3, \varepsilon)\}$
q_3	$\{(q_3, \varepsilon)\}$			$\{(q_3, \varepsilon)\}$				$\{(q_4, \varepsilon)\}$		
q_4										

Input
Pop
State/
Push



PDA M_3 recognizing the language $\{ww^{\mathcal{R}} \mid w \in \{0,1\}^*\}$

In-class exercise:
Fill in the blanks

Flashback

DFA Computation Rules

Informally

Given
- A **DFA** (~ a "Program")
- and **Input = string of chars,** e.g. "`1101`"

A **DFA computation** (~ "Program run"):
- **Start** in ***start state***

- **Repeat:**
  - **Read 1 char** from **Input,** and
  - **Change state** according to ***transition rules***

Result of **computation:**
- **Accept** if **last state** is ***Accept state***
- **Reject** otherwise

Formally (i.e., mathematically)

- $M =$ `mk-DFA`$(Q, \Sigma, \delta, q_0, F)$ ...
- $w = w_1 w_2 \cdots w_n$

A **DFA computation** is a **sequence of states:**

- specified by $\hat{\delta}(q_0, w)$ where:

- M **accepts** w if $\hat{\delta}(q_0, w) \in F$
- M **rejects** otherwise

Flashback

DFA Computation Rules

Informally

Given

- A **DFA** (~ a "Program")
- and **Input = string of chars**, e.g. "1101"

A **DFA** computation (~ "Program run"):

- Start in *start state*

- Repeat:
  - Read **1 char** from **Input,** and
  - Change state according to *transition rules*

Result of **computation:**

- Accept if **last state** is *Accept state*
- Reject otherwise

Formally (i.e., mathematically)

- M = **mk-DFA**$(\ Q,\ \Sigma,\ \delta,\ q_0,\ F\ )$ ...
- $w = w_1 w_2 \cdots w_n$

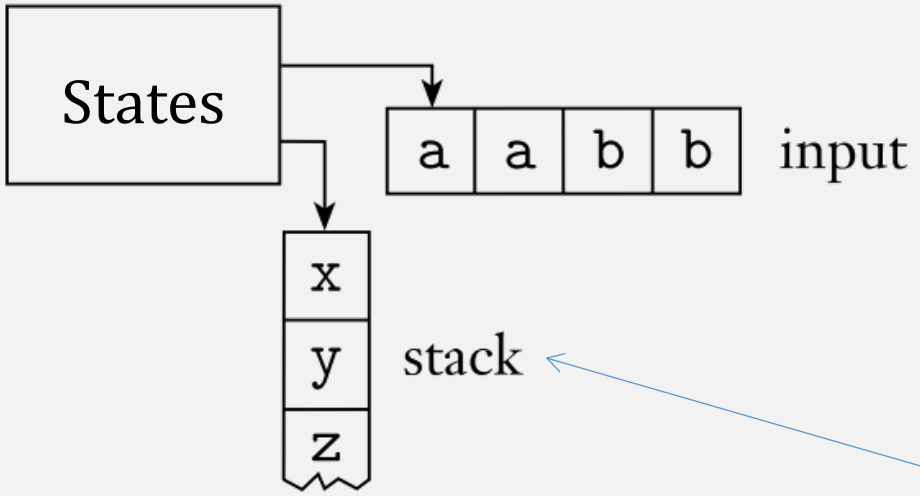
A **DFA computation** is a sequence of states:

- specified by $\hat{\delta}(q_0, w)$ where:

- M **accepts** w if $\hat{\delta}(q_0, w) \in F$
- M **rejects** otherwise

PDA Computation?

- **PDA** = NFA + a **stack**
  - Infinite memory
  - Push/pop top location only



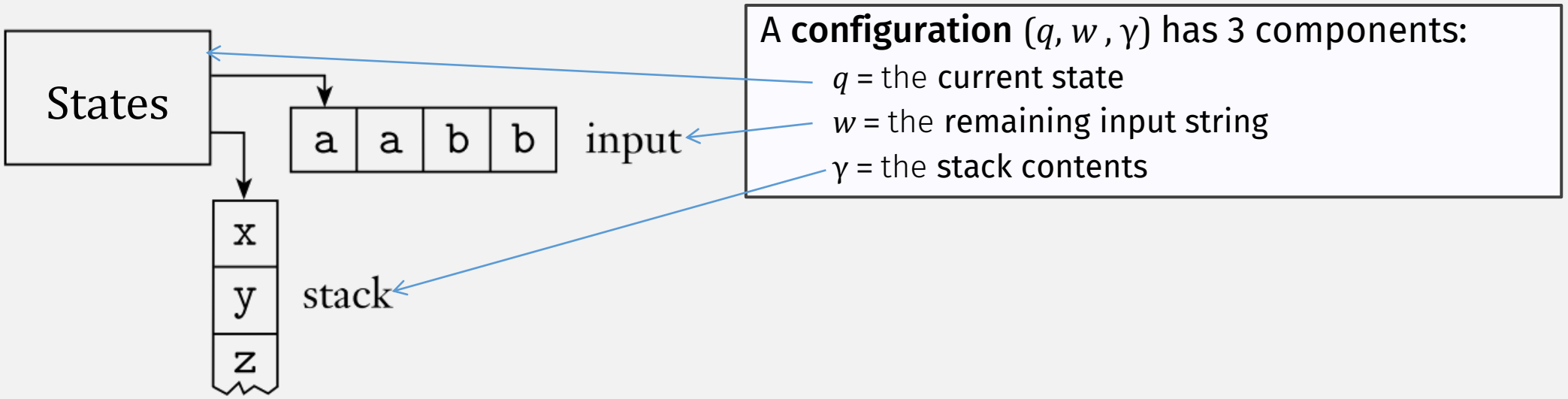
A DFA **computation** is a sequence of states …

A PDA **computation** is not just a sequence of states …

… because the **stack contents** can change too!

PDA Configurations (IDs)

- A **configuration** (or **ID**) is a "snapshot" of a **PDA's computation**



A **sequence of configurations** represents a **PDA** computation

PDA Computation, Formally

(one path in computation tree)

let P = **mk-PDA** (Q, Σ, Γ, δ, q_0, F)
where …

Single-step

Before / After **configurations**

$$(q_1, aw, X\beta) \vdash (q_2, w, \alpha\beta)$$

Read Input | Pop | Less 1 char | Push

if $\delta(q_1, a, X)$ contains (q_2, α)

$q_1, q_2 \in Q$

$a \in \Sigma \quad w \in \Sigma^*$

$X \in \Gamma \quad \beta, \alpha \in \Gamma^*$

A **configuration** (q, w, γ) has three components

- q = the **current state**
- w = the **remaining input string**
- γ = the **stack contents**

Multi-step

- Base Case — **0 steps**

$$I \vdash^* I \text{ for any ID } I$$

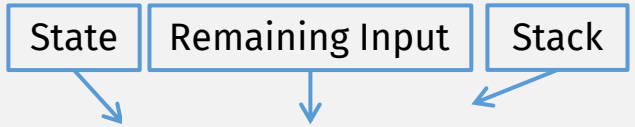
- Recursive Case — **1 or more steps**

$$I \vdash^* J \text{ if there exists some ID } K \text{ such that } I \vdash K \text{ and } K \vdash^* J$$

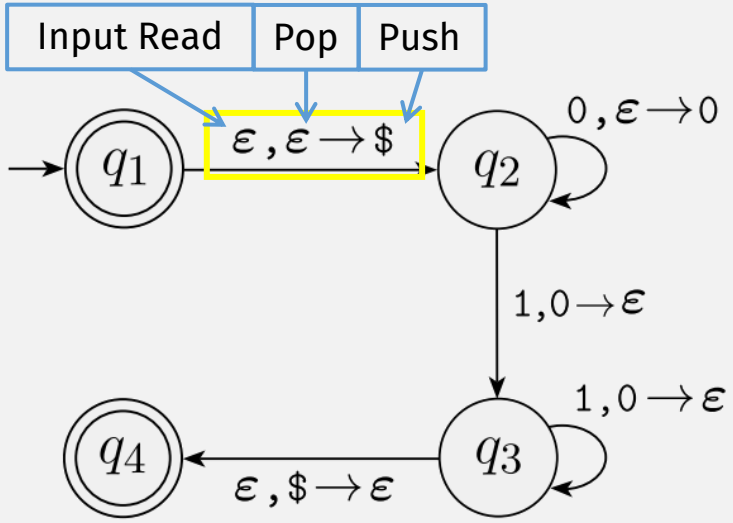
Single step | Recursive "call"

This specifies the **sequence of configurations** for a **PDA** computation

PDA Running Input String Example



$$(q_1, 0011, \varepsilon)$$



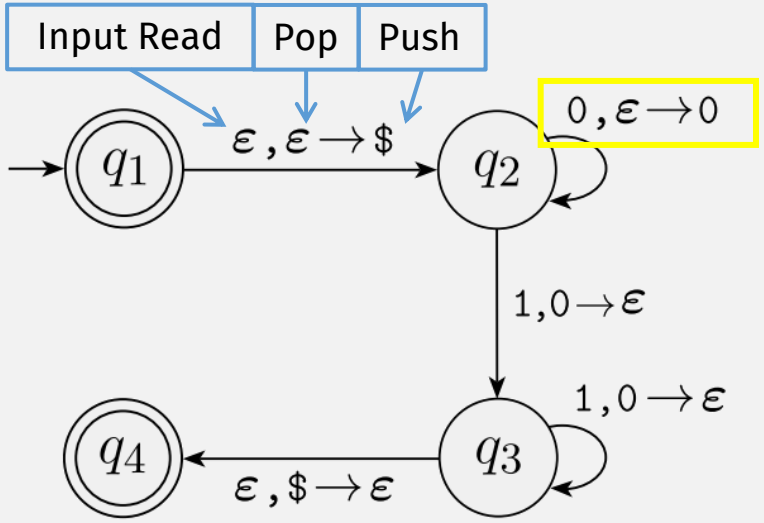
PDA Running Input String Example

State | Remaining Input | Stack

$$(q_1, 0011, \varepsilon) \vdash (q_2, 0011, \$)$$

$$\vdash (q_2, 011, 0\$)$$

Read 0, push 0

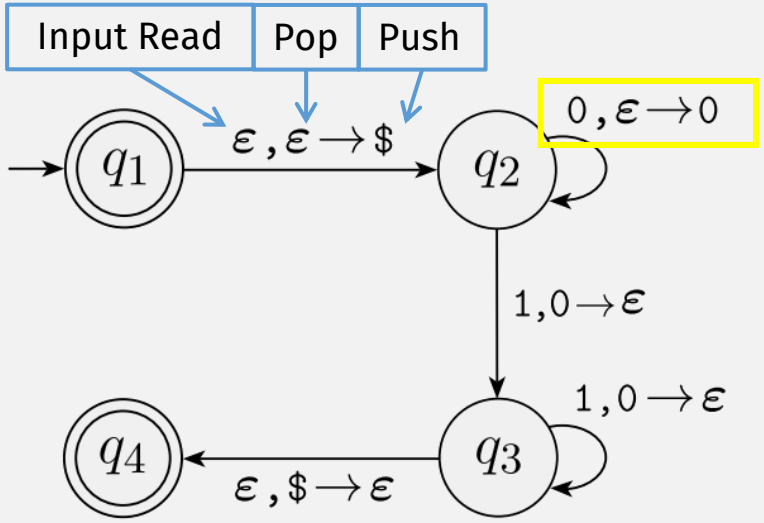


PDA Running Input String Example

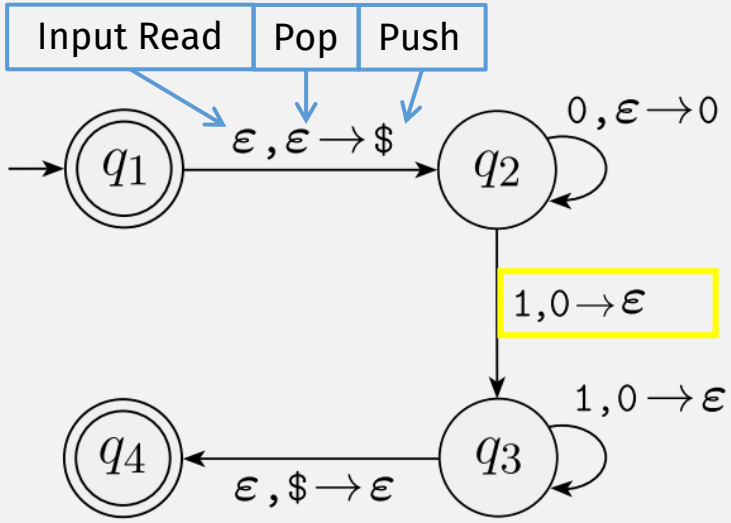
State | Remaining Input | Stack

$$(q_1, 0011, \varepsilon) \vdash (q_2, 0011, \$)$$
$$\vdash (q_2, 011, 0\$)$$
$$\vdash (q_2, 11, 00\$)$$

Read 0, push 0



PDA Running Input String Example

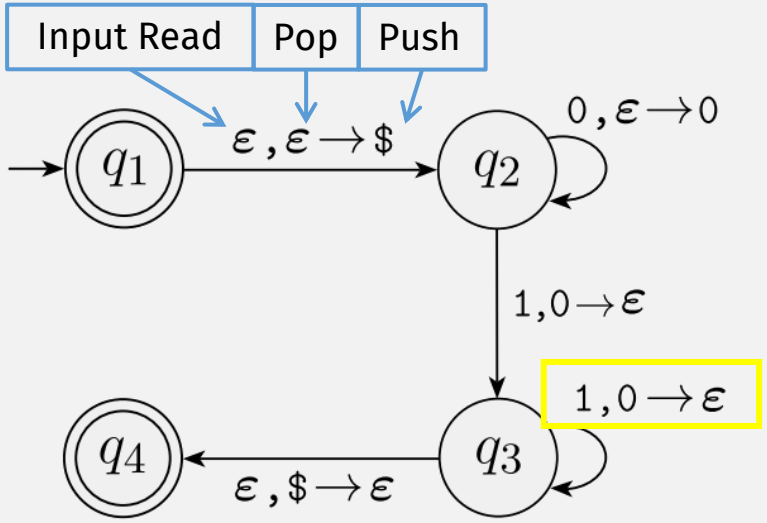


State | Remaining Input | Stack

$$(q_1, 0011, \varepsilon) \vdash (q_2, 0011, \$)$$
$$\vdash (q_2, 011, 0\$)$$
$$\vdash (q_2, 11, 00\$)$$
$$\vdash (q_3, 1, 0\$)$$

Read 1, pop 0

PDA Running Input String Example

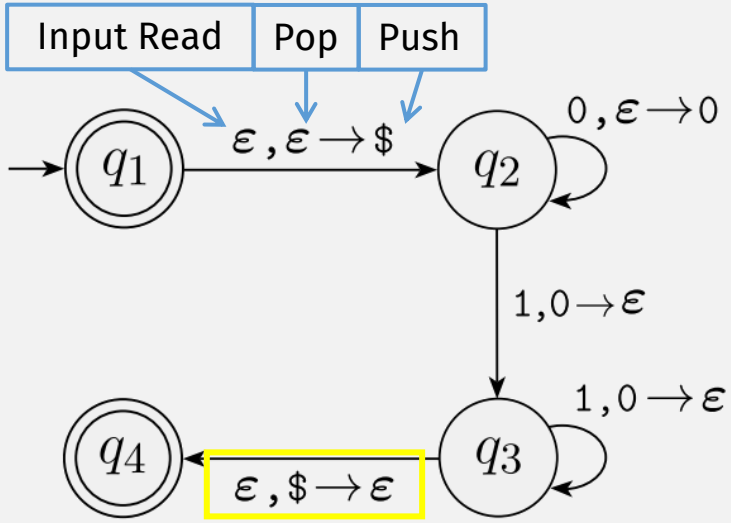


State | Remaining Input | Stack

$$(q_1, 0011, \varepsilon) \vdash (q_2, 0011, \$)$$
$$\vdash (q_2, 011, 0\$)$$
$$\vdash (q_2, 11, 00\$)$$
$$\vdash (q_3, 1, 0\$)$$
$$\vdash (q_3, \varepsilon, \$)$$

Read 1, pop 0

PDA Running Input String Example



State | Remaining Input | Stack

$$(q_1, 0011, \varepsilon) \vdash (q_2, 0011, \$)$$
$$\vdash (q_2, 011, 0\$)$$
$$\vdash (q_2, 11, 00\$)$$
$$\vdash (q_3, 1, 0\$)$$
$$\vdash (q_3, \varepsilon, \$)$$
$$\vdash (q_4, \varepsilon, \varepsilon)$$

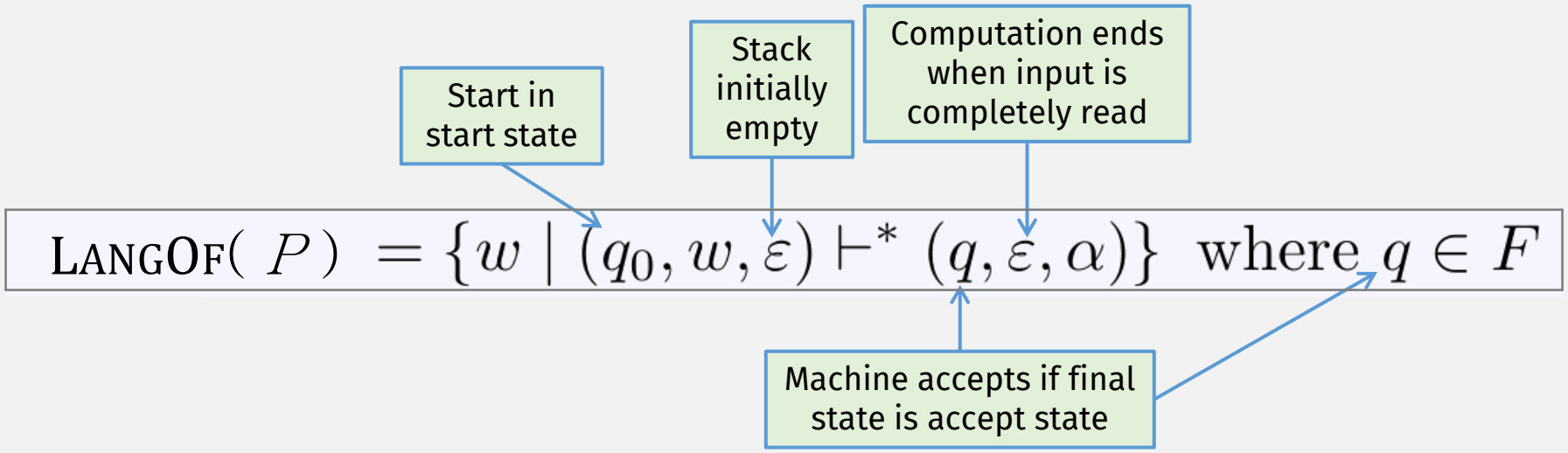
pop empty stack symbol

Flashback: Computation and Languages

- The **language** of a **machine** is the **set of all strings that it accepts**
- E.g., A **DFA** M **accepts** w **if** $\hat{\delta}(q_0, w) \in F$
- **Language** of M = $LangOf(M)$ = { w | M **accepts** w }

Language of a PDA

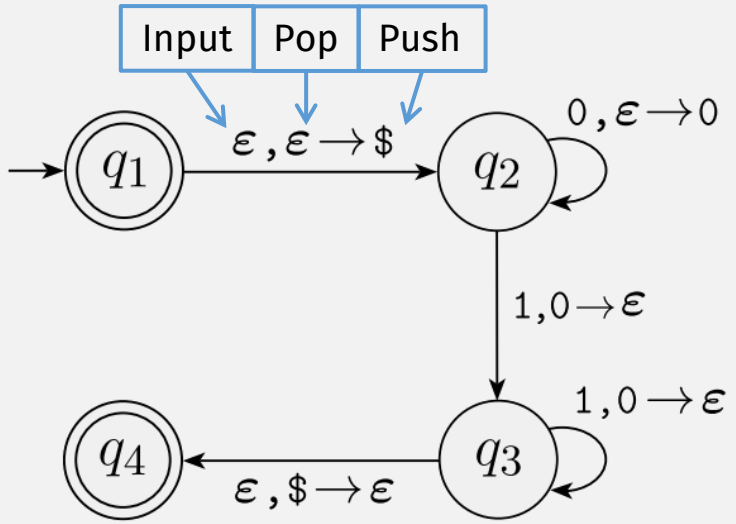
let P = **mk-PDA** (Q, Σ, Γ, δ, q_0, F)
where …



A **configuration** (q, w, γ) has three components

- q = the **current state**
- w = the **remaining input string**
- γ = the **stack contents**

PDAs and CFLs?



- **PDA** = NFA + a stack
  - Infinite memory
  - Push/pop top location only
- Want to prove: PDAs represent CFLs!
- We know: a CFL, by definition, is a language that is generated by a CFG
- Need to show: PDA ⇔ CFG
- Then, to prove that a language is a CFL, we can either:
  - Create a CFG, or
  - Create a PDA

Regular Language vs CFL Comparison

Regular Languages	Context-Free Languages (CFLs)
Regular Expression	Context-Free Grammar (CFG)
describes a **Regular Lang**	describes a **CFL**
Finite State Automaton (FSM)	**Push-down Automata** (PDA)
recognizes a **Regular Lang**	recognizes a **CFL**
Proved:	Must Prove:
Regular Lang ⇔Regular Expr ☑	CFL ⇔ PDA ???

thm

def

def

thm

L IS-A **CFL** CFLIFFPDA

IF AND ONLY IF

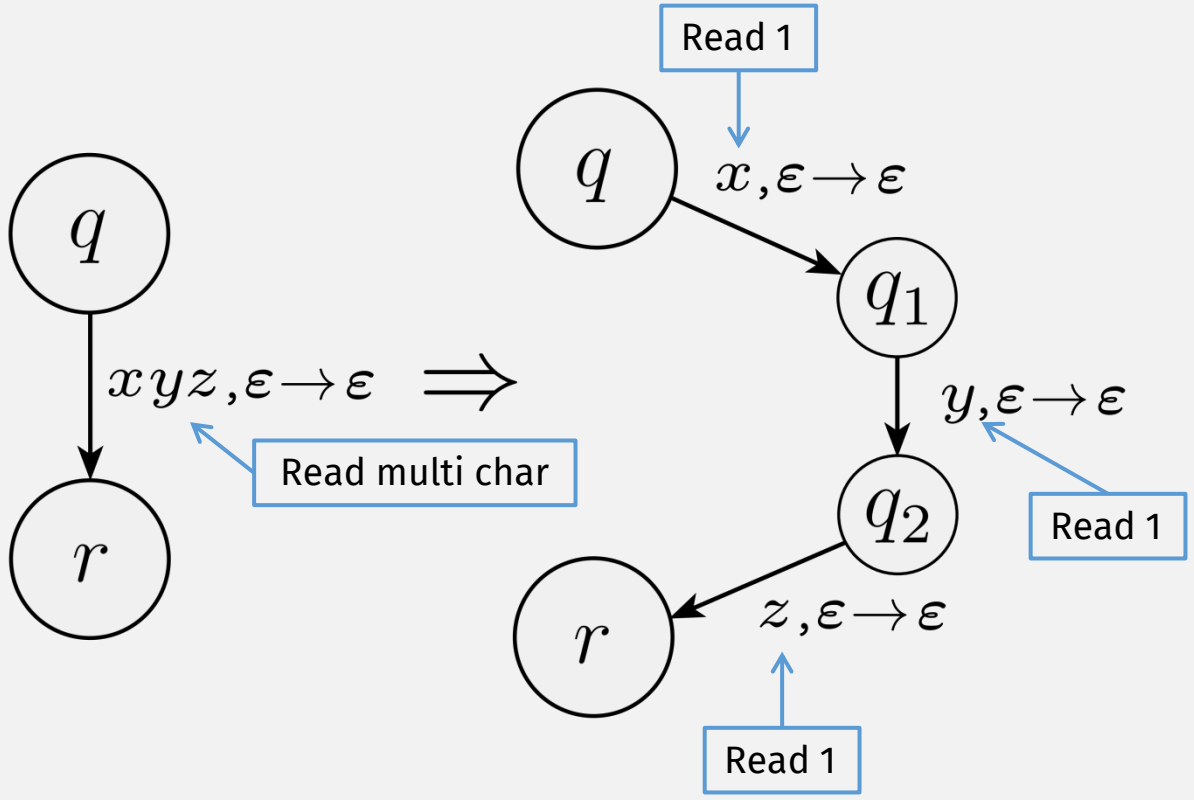
P IS-A **PDA** Machine AND P **recognizes** L

⇒ IF L IS-A **CFL** THEN P IS-A **PDA** Machine AND P **recognizes** L

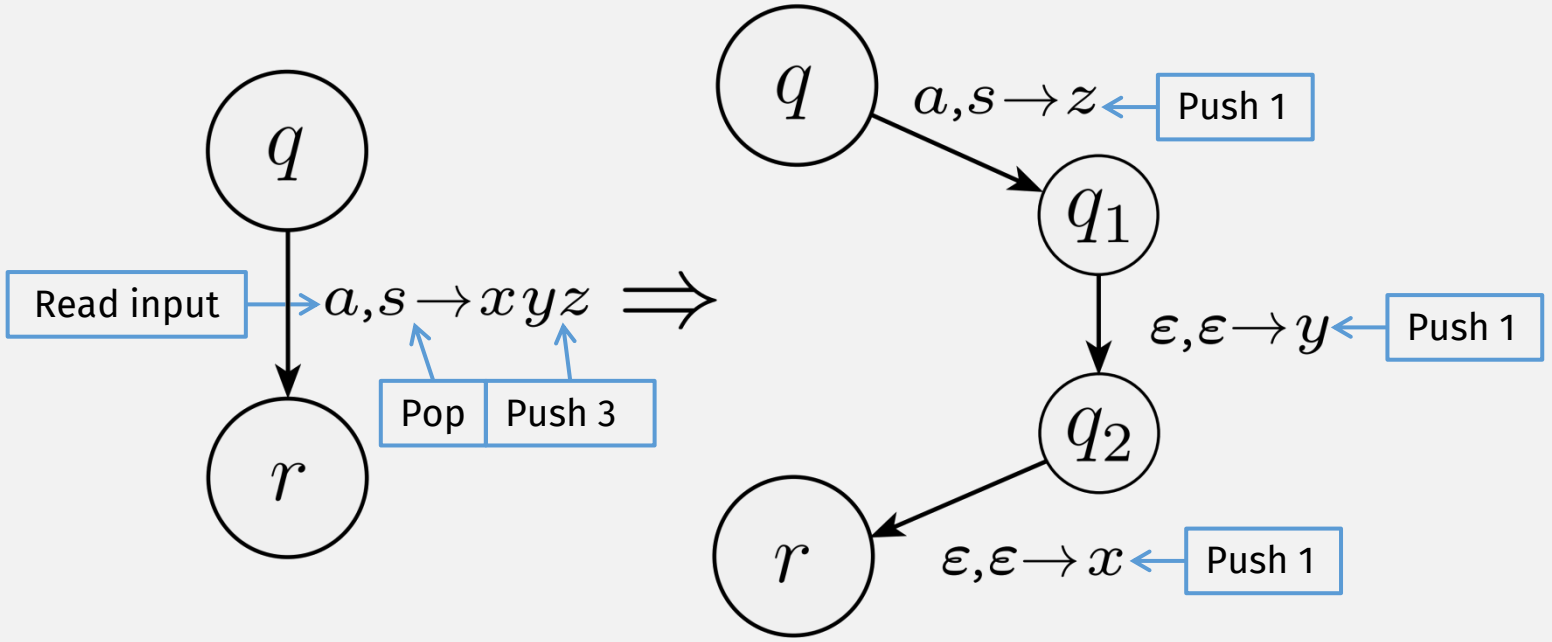
- We know: A **CFL** has a **CFG describing it** (definition of CFL)
- To prove this part, show: the **CFG** has an **equivalent PDA**

⇐ IF P IS-A **PDA** Machine AND P **recognizes** L THEN L IS-A **CFL**

Shorthand: Multi-Symbol Read Transition



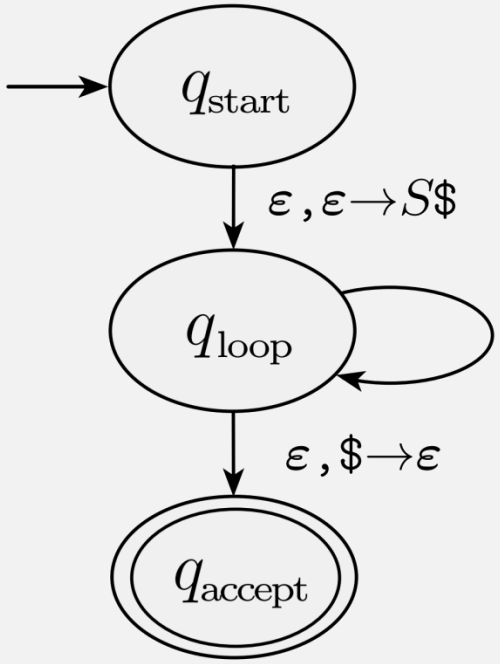
Shorthand: Multi-Stack Push Transition



Note the reverse order of pushes

CFG→PDA (sketch)

- **Construct PDA from CFG** such that:
  - **PDA accepts input only if CFG generates it**
- **PDA:**
  - **simulates generating a string with CFG rules**
  - **by** (nondeterministically) **trying all rules** to find the right ones

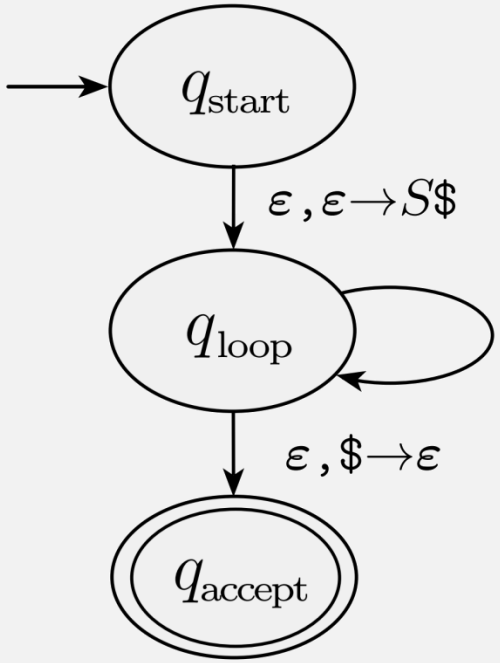


$\varepsilon, A \to w$ for rule $A \to w$

$\mathtt{a}, \mathtt{a} \to \varepsilon$ for terminal a

CFG→PDA (sketch)

- Construct PDA from CFG such that:
  - PDA accepts input only if CFG generates it
- PDA:
  - simulates generating a string with CFG rules
  - by (nondeterministically) trying all rules to find the right ones



Convert: every CFG rule to PDA "loop" transition that:
- Pops LHS variable
- Pushes RHS

(Stack is "workspace" containing intermediate string of vars + terminals)

$\varepsilon, A \to w$ for rule $A \to w$

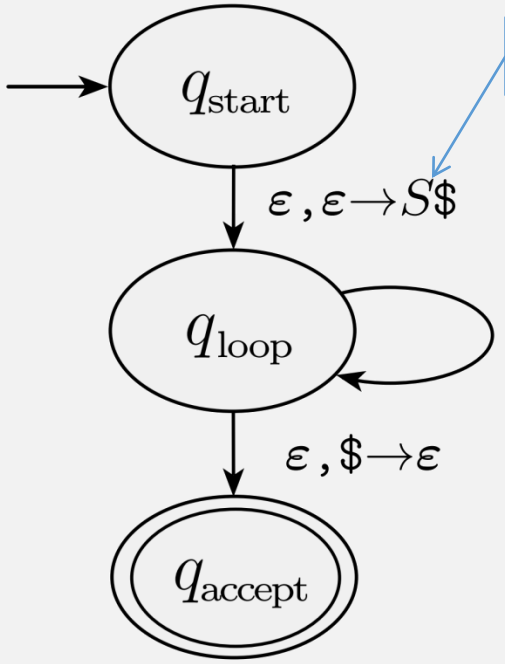
$a, a \to \varepsilon$ for terminal a

Convert: every terminal to "loop" transition that:
- Reads input char
- Pops matching char on stack

(Read the terminals as they become known)

CFG→PDA (sketch)

- Construct PDA from CFG such that:
  - PDA accepts input only if CFG generates it
- PDA:
  - simulates generating a string with CFG rules
  - by (nondeterministically) trying all rules to find the right ones



push **start variable** onto stack

If: stack top is **variable** A, pop and …

… push **rule's right-sides** (nondeterministically)

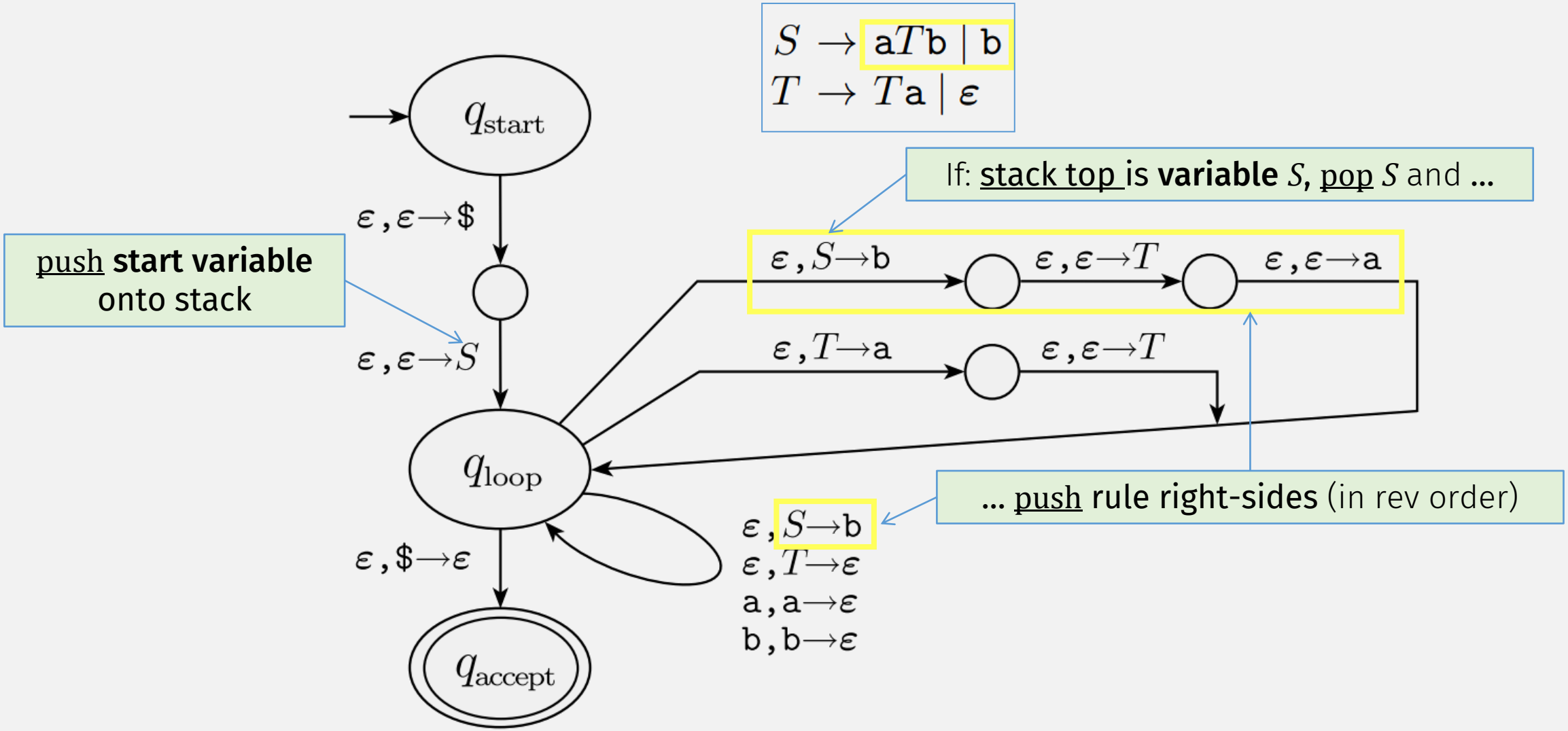
$\varepsilon, A \to w$ for rule $A \to w$

$\mathtt{a}, \mathtt{a} \to \varepsilon$ for terminal a

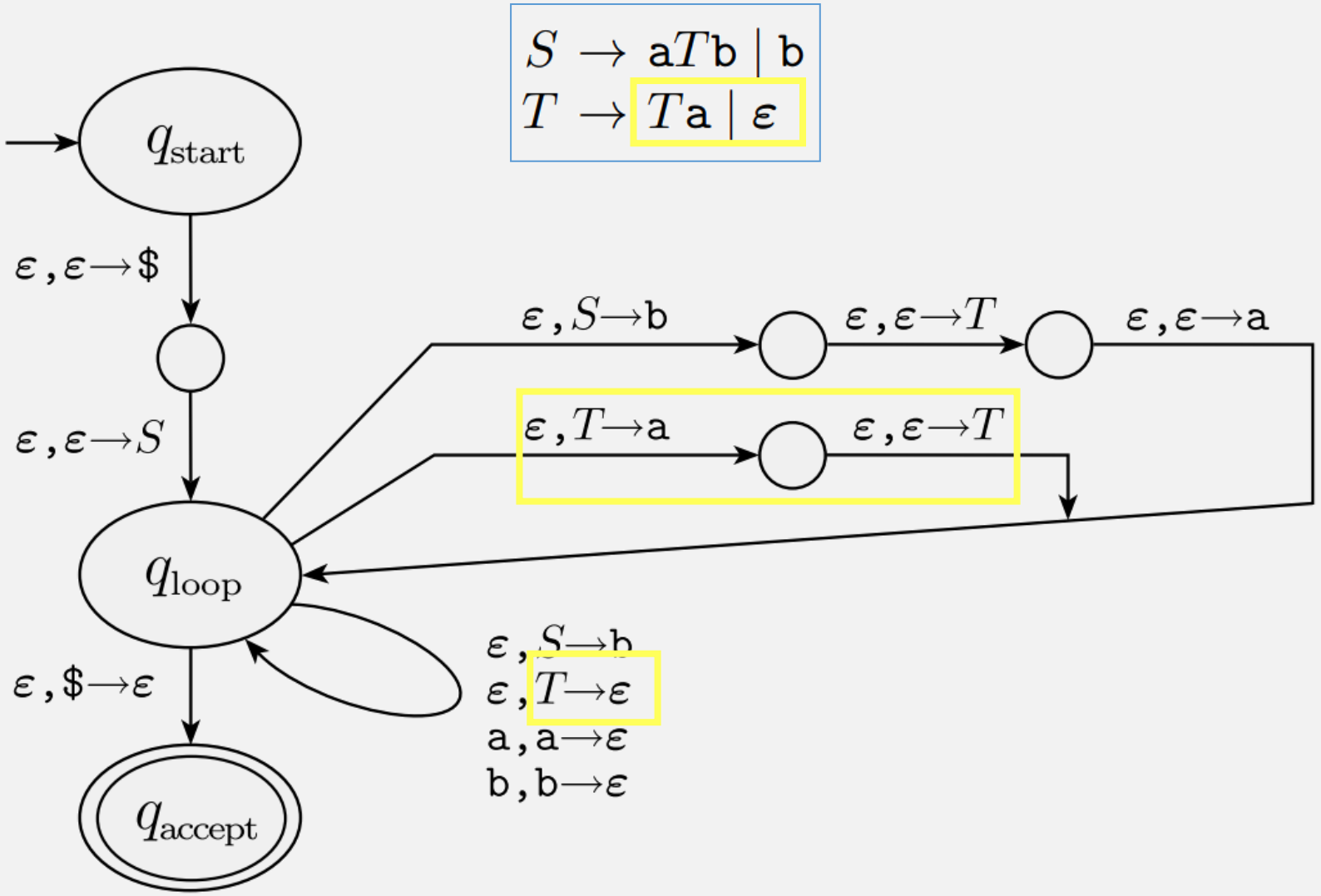
If: stack top is **terminal** a, pop and …

… read matching input

Example **CFG→PDA**

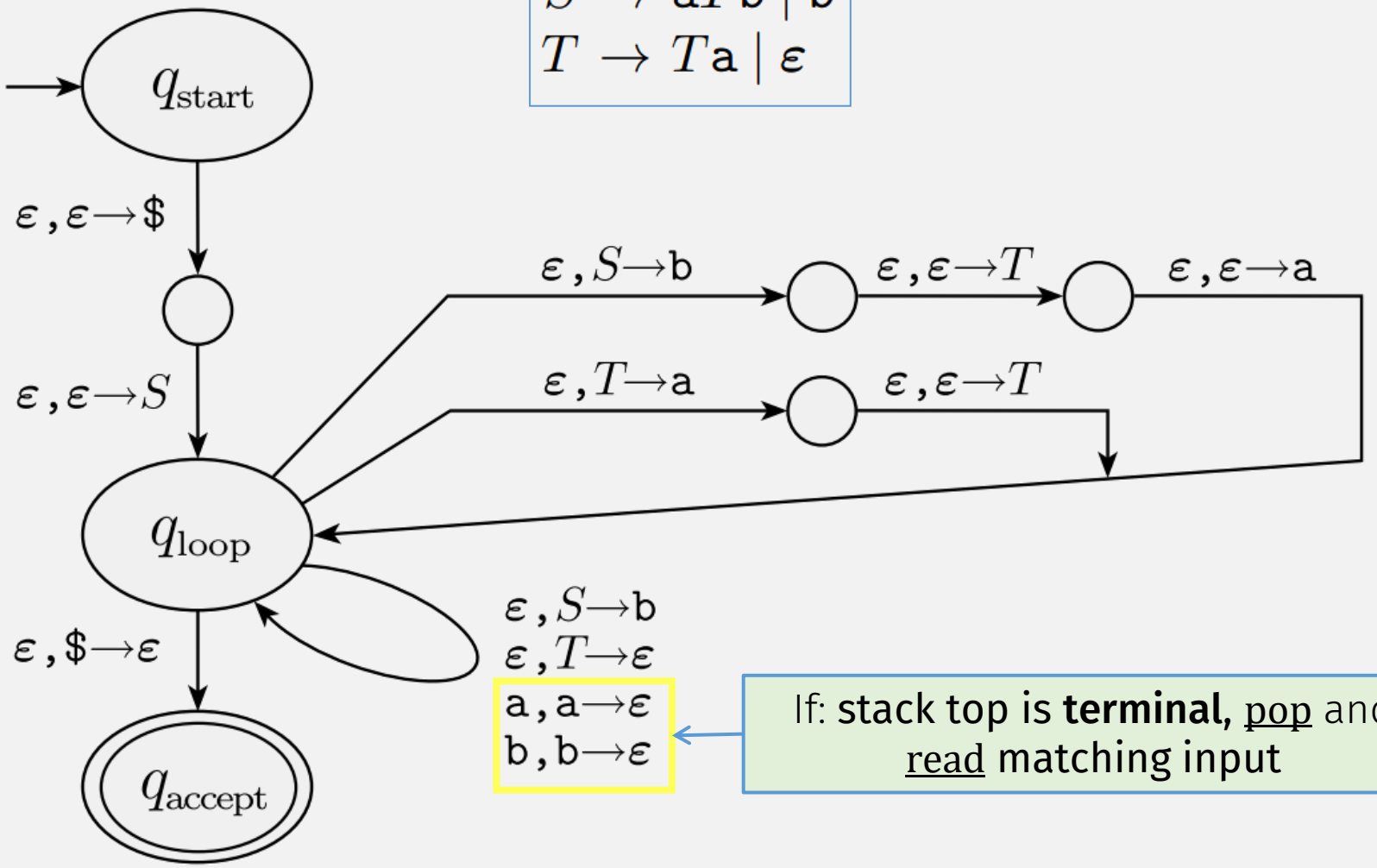


Example **CFG→PDA**



Example **CFG→PDA**

$S \to \mathtt{a}T\mathtt{b} \mid \mathtt{b}$
$T \to T\mathtt{a} \mid \varepsilon$



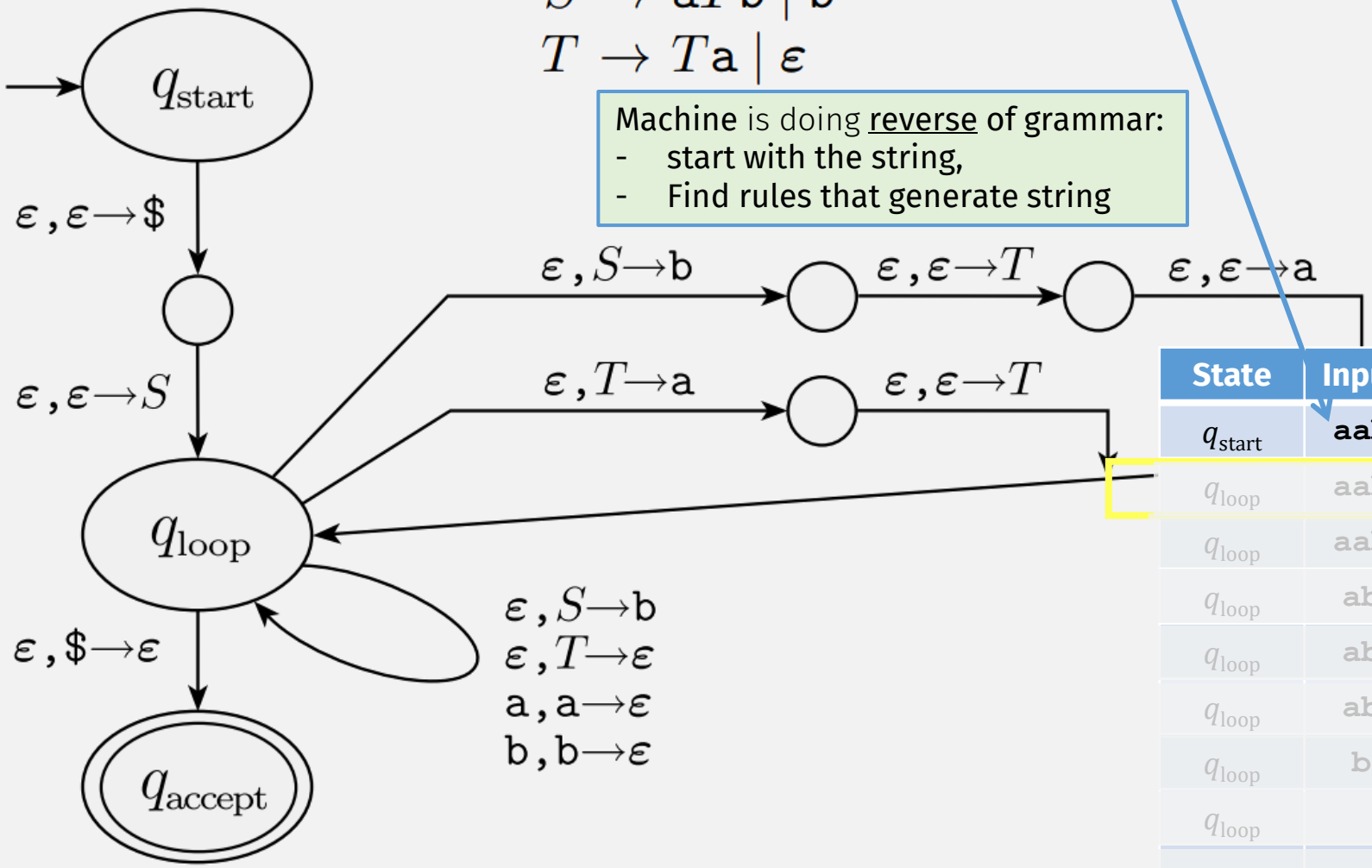
If: stack top is **terminal,** pop and read matching input

Example **CFG→PDA**

Example Derivation using CFG:
$S \Rightarrow \mathtt{a}T\mathtt{b}$ (using rule $S \to \mathtt{a}T\mathtt{b}$)
$\Rightarrow \mathtt{a}T\mathtt{ab}$ (using rule $T \to T\mathtt{a}$)
$\Rightarrow \mathtt{aab}$ (using rule $T \to \varepsilon$)

$S \to \mathtt{a}T\mathtt{b} \mid \mathtt{b}$
$T \to T\mathtt{a} \mid \varepsilon$

Machine is doing reverse of grammar:
- start with the string,
- Find rules that generate string



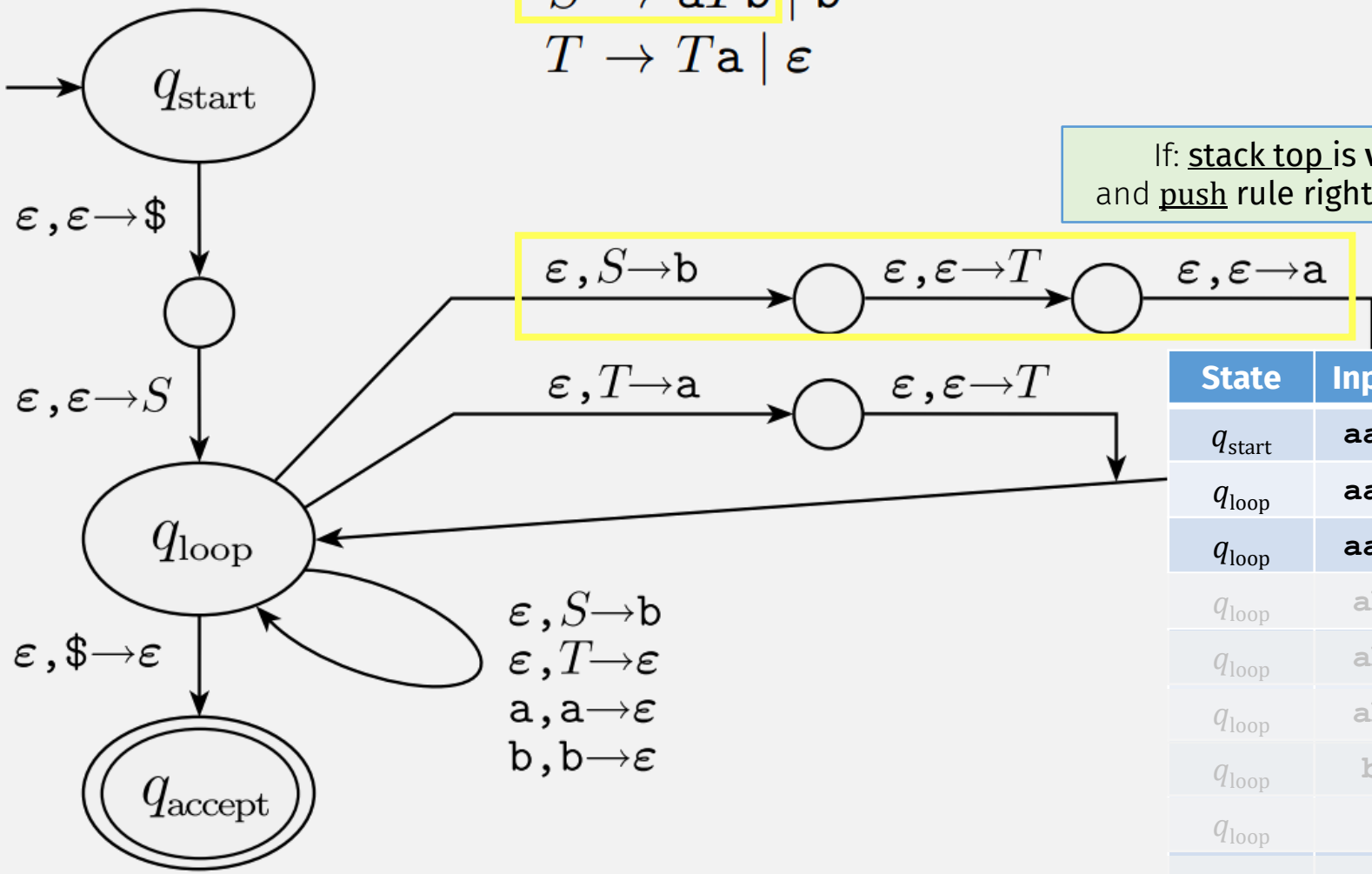
PDA Example

State	Input	Stack	Equiv Rule
q_{start}	aab		
q_{loop}	aab	$S\$$	
q_{loop}	aab	$\mathtt{a}T\mathtt{b}\$$	$S \to \mathtt{a}T\mathtt{b}$
q_{loop}	ab	$T\mathtt{b}\$$	
q_{loop}	ab	$T\mathtt{ab}\$$	$T \to T\mathtt{a}$
q_{loop}	ab	$\mathtt{ab}\$$	$T \to \varepsilon$
q_{loop}	b	$\mathtt{b}\$$	
q_{loop}		$\$$	
q_{accept}			

Example **CFG→PDA**

Example Derivation using CFG:
$S \Rightarrow \mathtt{a}T\mathtt{b}$ (using rule $S \to \mathtt{a}T\mathtt{b}$)
$\Rightarrow \mathtt{a}T\mathtt{ab}$ (using rule $T \to T\mathtt{a}$)
$\Rightarrow \mathtt{aab}$ (using rule $T \to \varepsilon$)

$S \to \mathtt{a}T\mathtt{b} \mid \mathtt{b}$
$T \to T\mathtt{a} \mid \varepsilon$



If: stack top is **variable** S, pop S
and push **rule right-sides** (in rev order)

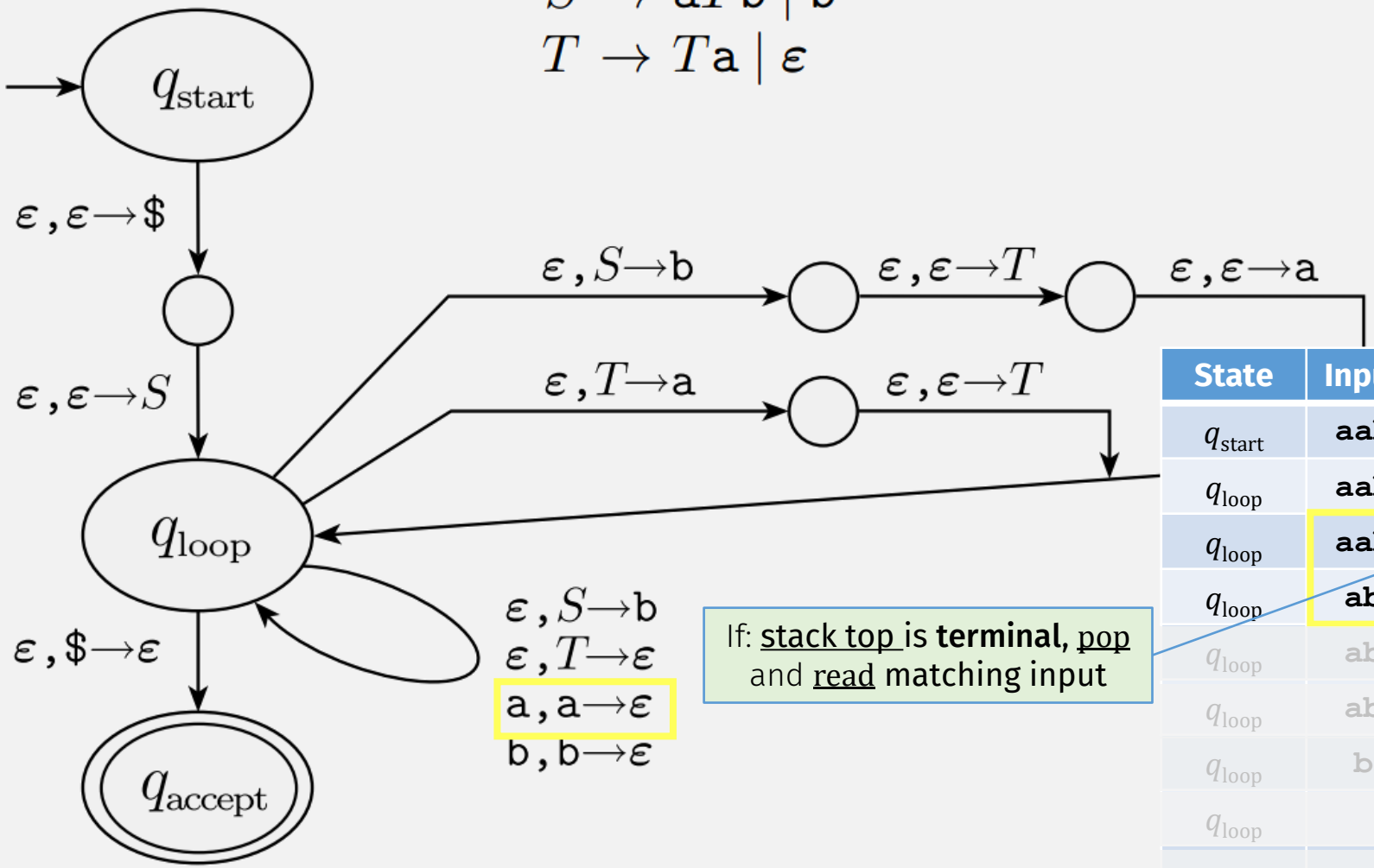
PDA Example

State	Input	Stack	Equiv Rule
q_{start}	aab		
q_{loop}	aab	$S\$$	
q_{loop}	aab	$\mathtt{a}T\mathtt{b}\$$	$S \to \mathtt{a}T\mathtt{b}$
q_{loop}	ab	$T\mathtt{b}\$$	
q_{loop}	ab	$T\mathtt{ab}\$$	$T \to T\mathtt{a}$
q_{loop}	ab	$\mathtt{ab}\$$	$T \to \varepsilon$
q_{loop}	b	$\mathtt{b}\$$	
q_{loop}		$\$$	
q_{accept}			

Example **CFG→PDA**

Example Derivation using CFG:
$S \Rightarrow \mathtt{a}T\mathtt{b}$ (using rule $S \to \mathtt{a}T\mathtt{b}$)
$\Rightarrow \mathtt{a}T\mathtt{ab}$ (using rule $T \to T\mathtt{a}$)
$\Rightarrow \mathtt{aab}$ (using rule $T \to \varepsilon$)

$$S \to \mathtt{a}T\mathtt{b} \mid \mathtt{b}$$
$$T \to T\mathtt{a} \mid \varepsilon$$



If: stack top is **terminal**, pop and read matching input

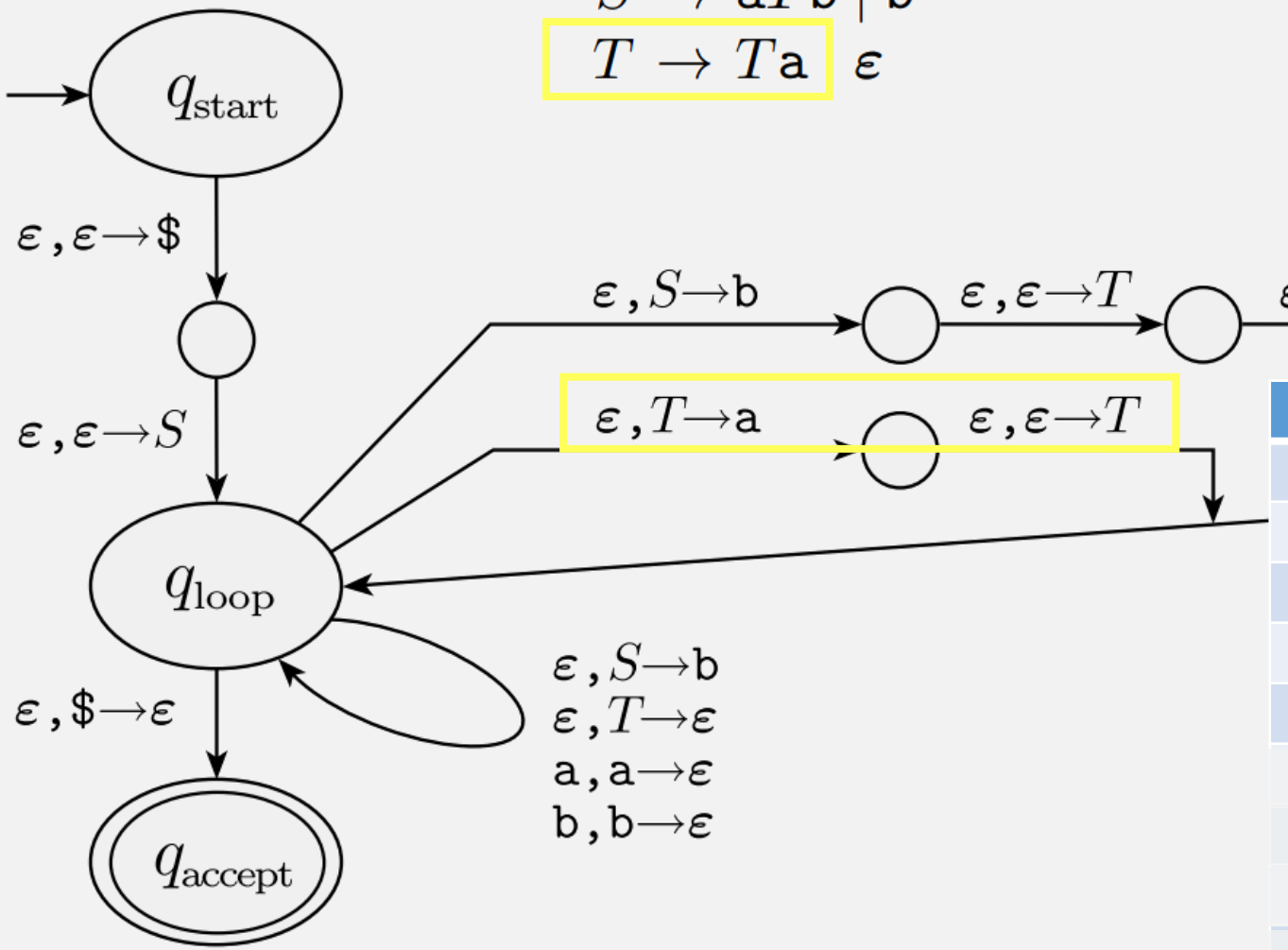
PDA Example

State	Input	Stack	Equiv Rule
q_{start}	**aab**		
q_{loop}	**aab**	S\$	
q_{loop}	**aab**	**a**T**b**\$	$S \to \mathtt{a}T\mathtt{b}$
q_{loop}	**ab**	T**b**\$	
q_{loop}	**ab**	T**ab**\$	$T \to T\mathtt{a}$
q_{loop}	**ab**	**ab**\$	$T \to \varepsilon$
q_{loop}	**b**	**b**\$	
q_{loop}		\$	
q_{accept}			

Example **CFG→PDA**

Example Derivation using CFG:
$S \Rightarrow \mathtt{a}T\mathtt{b}$ (using rule $S \to \mathtt{a}T\mathtt{b}$)
$\Rightarrow \mathtt{a}T\mathtt{ab}$ (using rule $T \to T\mathtt{a}$)
$\Rightarrow \mathtt{aab}$ (using rule $T \to \varepsilon$)

$$S \to \mathtt{a}T\mathtt{b} \mid \mathtt{b}$$
$$T \to T\mathtt{a} \mid \varepsilon$$



PDA Example

State	Input	Stack	Equiv Rule
q_{start}	aab		
q_{loop}	aab	$S\$$	
q_{loop}	aab	$\mathtt{a}T\mathtt{b}\$$	$S \to \mathtt{a}T\mathtt{b}$
q_{loop}	ab	$T\mathtt{b}\$$	
q_{loop}	ab	$T\mathtt{ab}\$$	$T \to T\mathtt{a}$
q_{loop}	ab	$\mathtt{ab}\$$	$T \to \varepsilon$
q_{loop}	b	$\mathtt{b}\$$	
q_{loop}		$\$$	
q_{accept}			

L IS-A **CFL** CFLIFFPDA

IF AND ONLY IF

P IS-A **PDA** Machine AND P **recognizes** L

☑ ⇒ IF L IS-A **CFL** THEN P IS-A **PDA** Machine AND P **recognizes** L

⇐ IF P IS-A **PDA** Machine AND P **recognizes** L THEN L IS-A **CFL**

To prove this part: show PDA has an equivalent CFG

PDA→CFG: Prelims

Before converting PDA to CFG, <u>modify</u> it so :

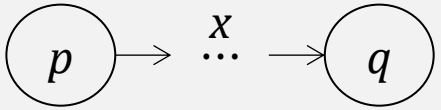
1. It has a single accept state, q_{accept}.
2. It empties its stack before accepting.
3. Each transition either pushes a symbol onto the stack (a *push* move) or pops one off the stack (a *pop* move), but it does not do both at the same time.

<u>Important</u>:
This doesn't change the language recognized by the PDA

PDA P -> CFG G: Transitions and Variables

$P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{\text{accept}}\})$ variables of G are $\{A_{pq} \mid p, q \in Q\}$

- Want: if P goes from state p to q reading input x, then some A_{pq} generates x



- So: For every pair of states p, q in P, add variable A_{pq} to G

- Then: connect the variables together by,
  - Add rules: $A_{pq} \rightarrow A_{pr}A_{rq}$, for each state r
  - These rules allow: grammar to simulate every possible transition
  - (We haven't added input read/generated terminals yet)

The Key IDEA

- To add terminals: pair up stack pushes and pops (essence of a CFL)

PDA P -> CFG G: Generating Strings

$P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{\text{accept}}\})$

variables of G are $\{A_{pq} \mid p, q \in Q\}$

- The key: **pair up stack pushes and pops** (essence of a CFL)

if $\delta(p, a, \varepsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ε),

put the rule $A_{pq} \rightarrow aA_{rs}b$ in G

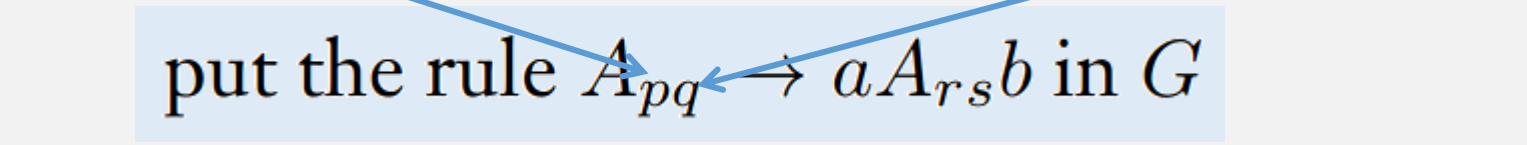
PDA P -> CFG G: Generating Strings

$P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{\text{accept}}\})$

variables of G are $\{A_{pq} \mid p, q \in Q\}$

- The key: **pair up stack pushes and pops** (essence of a CFL)

if $\delta(p, a, \varepsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ε),



put the rule $A_{pq} \rightarrow aA_{rs}b$ in G

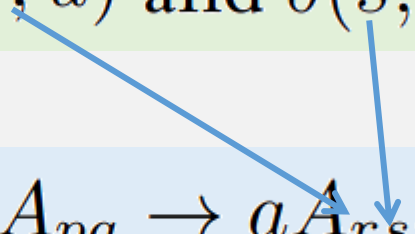
PDA P -> CFG G: Generating Strings

$P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{\text{accept}}\})$

variables of G are $\{A_{pq} \mid p, q \in Q\}$

- The key: **pair up stack pushes and pops** (essence of a CFL)

if $\delta(p, a, \varepsilon)$ contains (r, u) and $\delta(s, b, u)$ contains (q, ε),



put the rule $A_{pq} \rightarrow aA_{rs}b$ in G

L IS-A **CFL** CFLIFFPDA

IF AND ONLY IF

P IS-A **PDA** Machine AND P **recognizes** L

☑ ⇒ IF L IS-A **CFL** THEN P IS-A **PDA** Machine AND P **recognizes** L

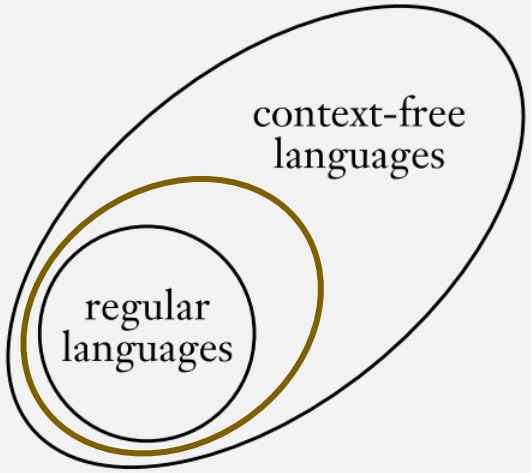
key step: **Convert CFG→PDA**

☑ ⇐ IF P IS-A **PDA** Machine AND P **recognizes** L THEN L IS-A **CFL**

key step: **Convert PDA→CFG**

■

Regular vs Context-Free Languages
(and others?)

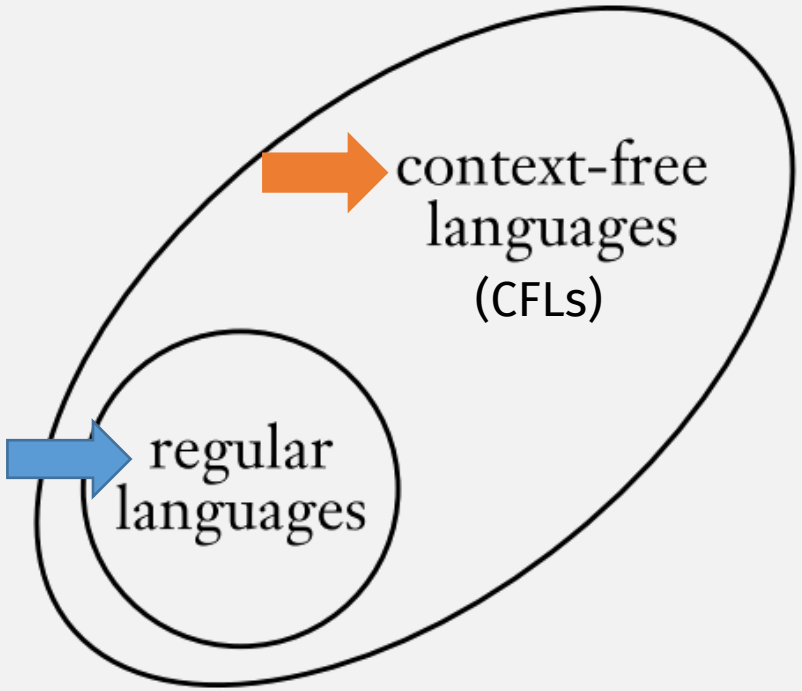


Is This Diagram “Correct”?

(What are the statements implied by this diagram?)

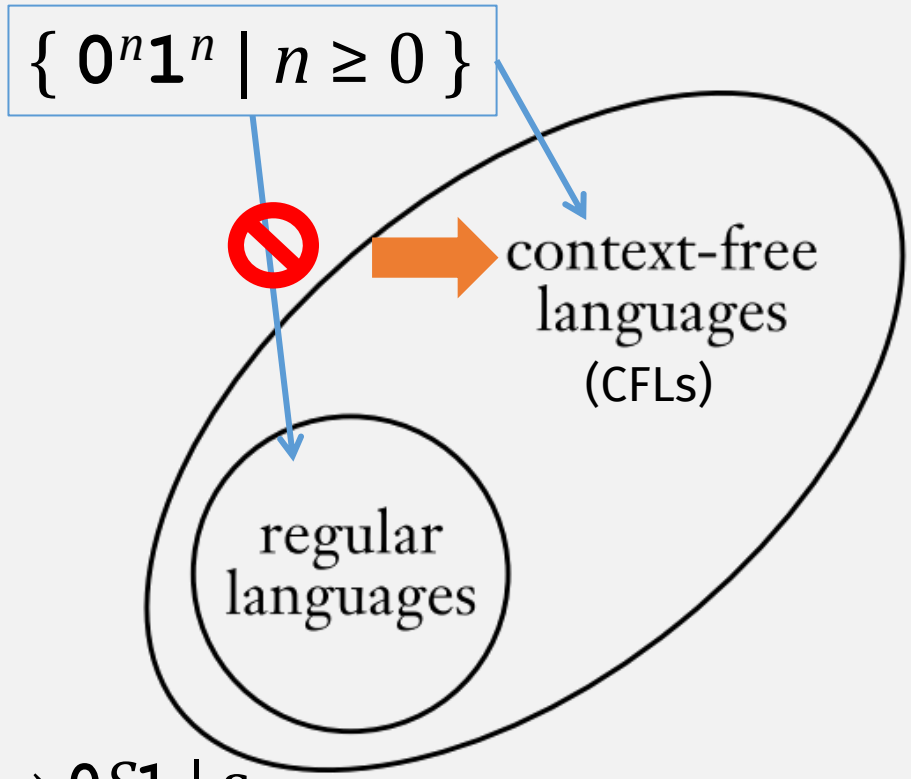
1. Every regular language is a CFL

2. Not every CFL is a regular language



How to Prove This Diagram "Correct"?

1. Every regular language is a CFL



2. Not every CFL is a regular language

Find a CFL that is not regular

$\{ \mathtt{0}^n\mathtt{1}^n \mid n \geq 0 \}$

- It's a CFL
  - *Proof summary*: CFG $S \rightarrow \mathtt{0}S\mathtt{1} \mid \varepsilon$
- It's not regular
  - *Proof summary*: by contradiction using the Pumping Lemma

How to Prove This Diagram "Correct"?

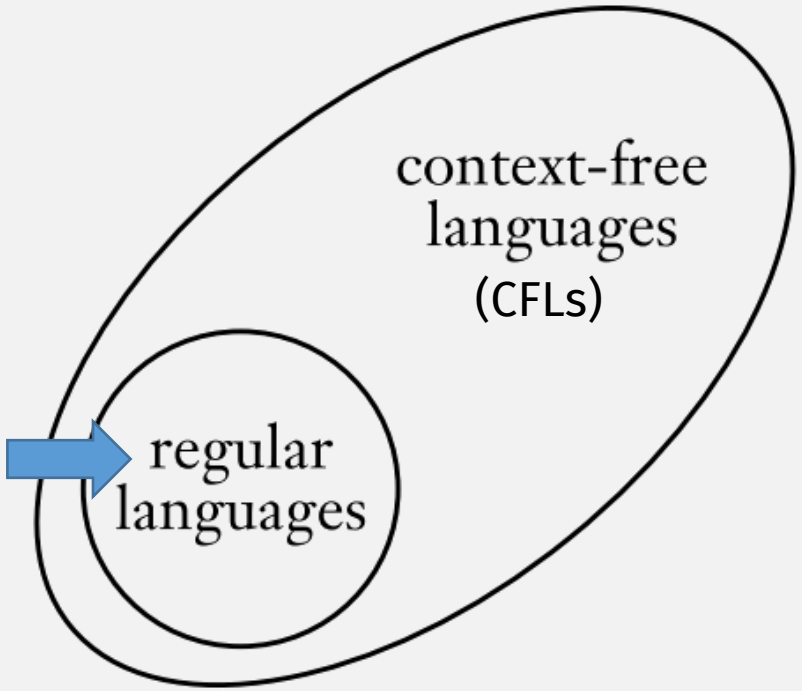
1. Every regular language is a CFL

For any regular language A, show …

… it has a CFG or PDA

☑ 2. Not every CFL is a regular language

A regular language is represented by a:
- DFA
- NFA
- Regular Expression



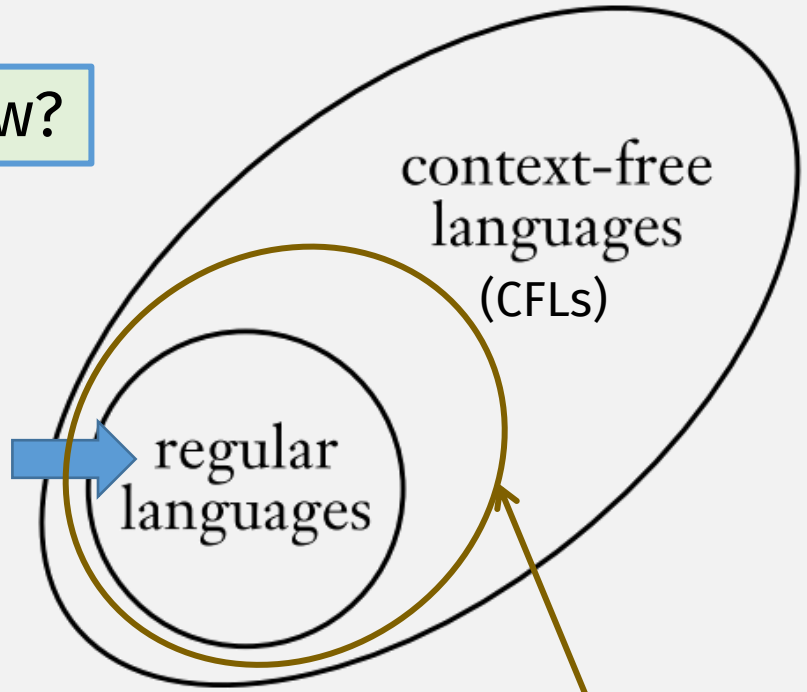
Regular Languages are CFLs: 3 Ways to Prove

- DFA → CFG or PDA

Coming soon to a future hw?

- NFA → CFG or PDA

- Regular expression → CFG or PDA



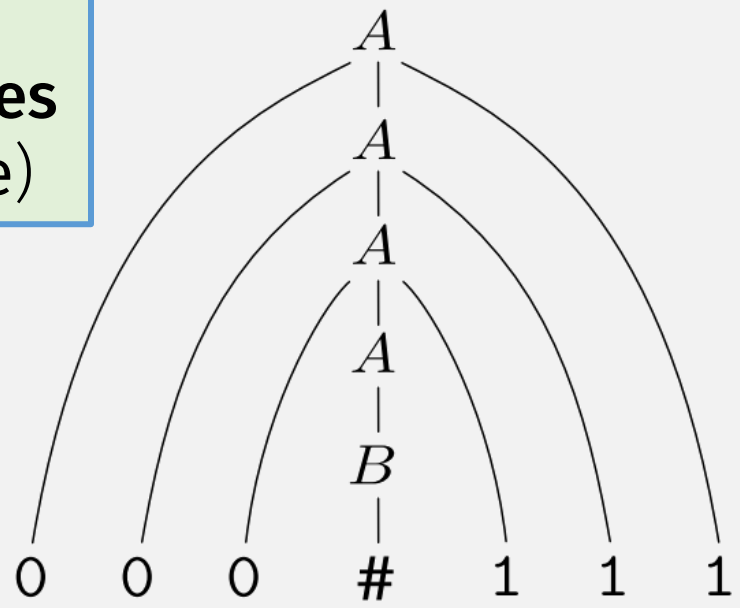
Are there other interesting subsets of CFLs?

Deterministic CFLs and DPDAs

Previously: Generating Strings

Generating strings:
1. Start with **start variable,**
2. Repeatedly *apply* **CFG rules**
to **get string** (and **parse tree**)

$$A \to 0A1$$
$$A \to B$$
$$B \to \#$$

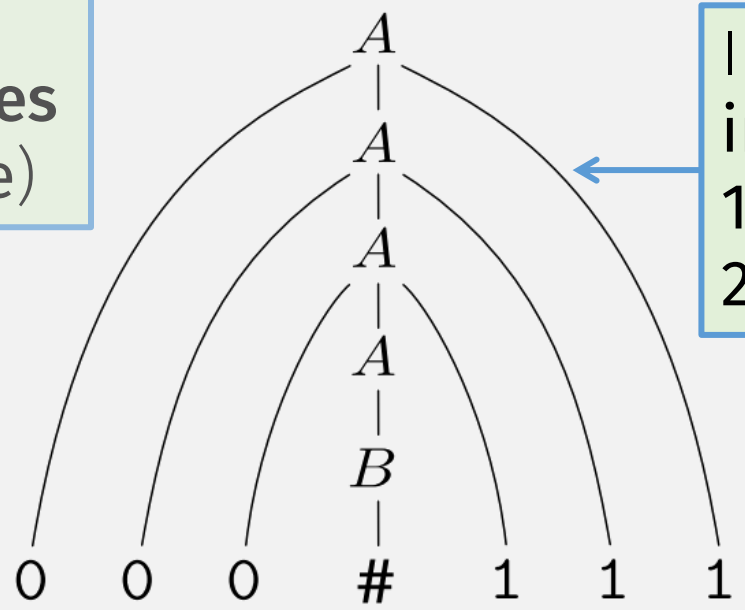


$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$$

Generating vs Parsing

Generating strings:
1. Start with **start variable,**
2. Repeatedly *apply* **CFG rules**
to get string (and **parse tree**)

$$A \to 0A1$$
$$A \to B$$
$$B \to \#$$



In practice, opposite is more interesting:
1. Start with **string,**
2. Then **parse** into **parse tree**

$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$$

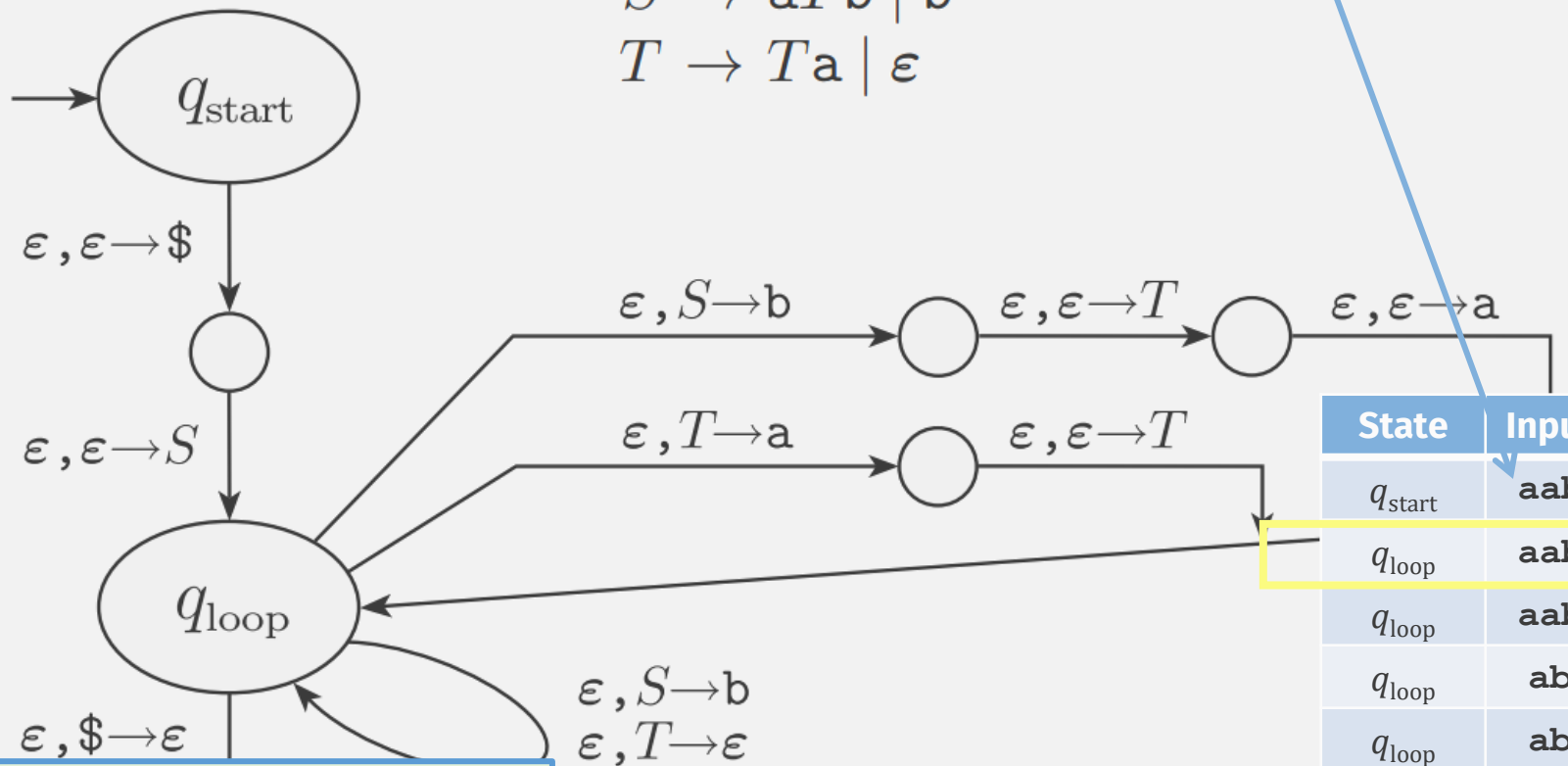
Generating vs Parsing

- In practice, **parsing** a string more important than **generating** one
  - E.g., a **compiler** (first) parses source code into a parse tree
  - (Actually, *any* program with string inputs must first parse it)

Previously: Example **CFG→PDA**

Example Derivation using CFG:
$S \Rightarrow \mathtt{a}T\mathtt{b}$ (using rule $S \to \mathtt{a}T\mathtt{b}$)
$\Rightarrow \mathtt{a}T\mathtt{ab}$ (using rule $T \to T\mathtt{a}$)
$\Rightarrow \mathtt{aab}$ (using rule $T \to \varepsilon$)

$$S \to \mathtt{a}T\mathtt{b} \mid \mathtt{b}$$
$$T \to T\mathtt{a} \mid \varepsilon$$



PDA Example

State	Input	Stack	Equiv Rule
q_{start}	aab		
q_{loop}	aab	$S\$$	
q_{loop}	aab	$a T b\$$	$S \to \mathtt{a}T\mathtt{b}$
q_{loop}	ab	$T b\$$	
q_{loop}	ab	$T a b\$$	$T \to T\mathtt{a}$
q_{loop}	ab	$ab\$$	$T \to \varepsilon$
q_{loop}	b	$b\$$	
q_{loop}		$\$$	
q_{accept}			

This Machine is **parsing**:
1. Start with (input) string,
2. Find **rules** that **generate** string

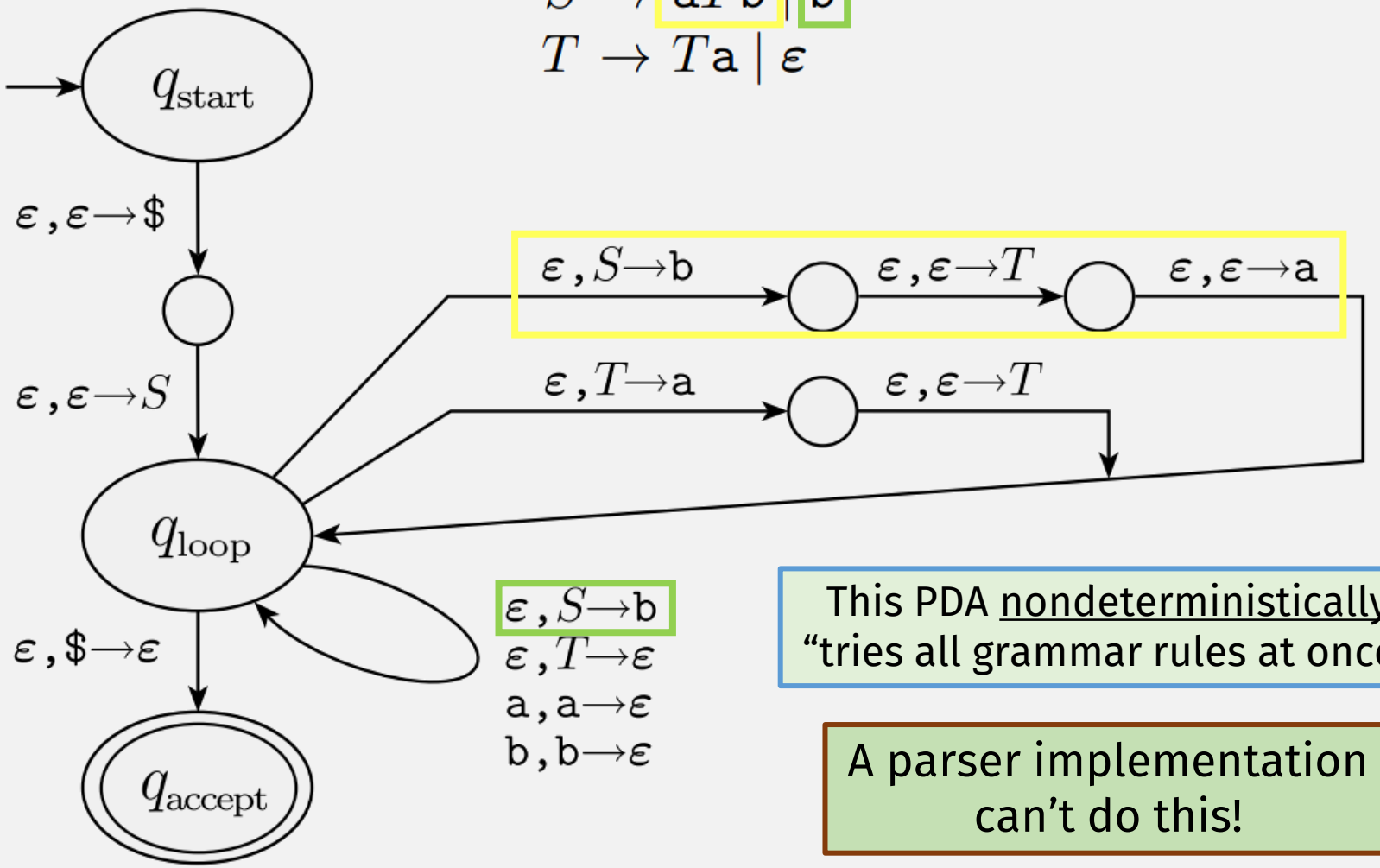
Generating vs Parsing

- In practice, **parsing** a string more important than **generating** one
  - E.g., a **compiler** (first step) parses source code into a parse tree
  - (Actually, *any* program with string inputs must first parse it)

- But: the PDAs we've seen are non-deterministic (like NFAs)

Previously: (Nondeterministic) PDA

$S \to \mathtt{a}T\mathtt{b} \mid \mathtt{b}$
$T \to T\mathtt{a} \mid \varepsilon$



This PDA nondeterministically "tries all grammar rules at once"

A parser implementation can't do this!

Generating vs Parsing

- In practice, **parsing** a string more important than **generating** one
  - E.g., a **compiler** (first step) parses source code into a parse tree
  - (Actually, *any* program with string inputs must first parse it)

- But: the PDAs we've seen are non-deterministic (like NFAs)

- Compiler's parsing algorithm must be deterministic

- So: to model parsers, we need a **Deterministic PDA** (DPDA)

DPDA: Formal Definition

The language of a DPDA is called a ***deterministic context-free language***.

A ***deterministic pushdown automaton*** is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$, where Q, Σ, Γ, and F are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow (Q \times \Gamma_\varepsilon) \cup \{\emptyset\}$ is the transition function
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

"do nothing"

A ***pushdown automaton*** is a 6-tuple

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta: Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \longrightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Difference: DPDA has only one possible action, for any given state, input, and stack op (similar to DFA vs NFA)

Must take into account ε reads or stack ops!
E.g., if $\delta(q, a, X)$ does "something",
then $\delta(q, \varepsilon, X)$ must "do nothing"

DPDAs are Not Equivalent to PDAs!

$$R \to S \mid T$$
$$S \to \mathtt{a}S\mathtt{b} \mid \mathtt{ab}$$
$$T \to \mathtt{a}T\mathtt{bb} \mid \mathtt{abb}$$

- A PDA can non-deterministically "try all rules" (abandoning failed attempts);
- A DPDA must choose one rule at each step! (cant go back after reading input!)

used S rule

Parsing = deriving reversed: start with string, end with parse tree

$\mathtt{aaa}\underline{\mathtt{a}}\mathtt{bbb} \rightarrowtail \mathtt{a}\underline{\mathtt{a}}\mathtt{S}\underline{\mathtt{b}}\mathtt{b}$

When parsing this string, when does it know which rule was used, S or T?

used T rule

$\mathtt{aaa}\underline{\mathtt{a}}\mathtt{bbbbbb} \rightarrowtail \mathtt{a}\underline{\mathtt{a}}\mathtt{T}\underline{\mathtt{b}}\mathtt{bbb}$

Choosing "correct" rule depends on rest of the input!

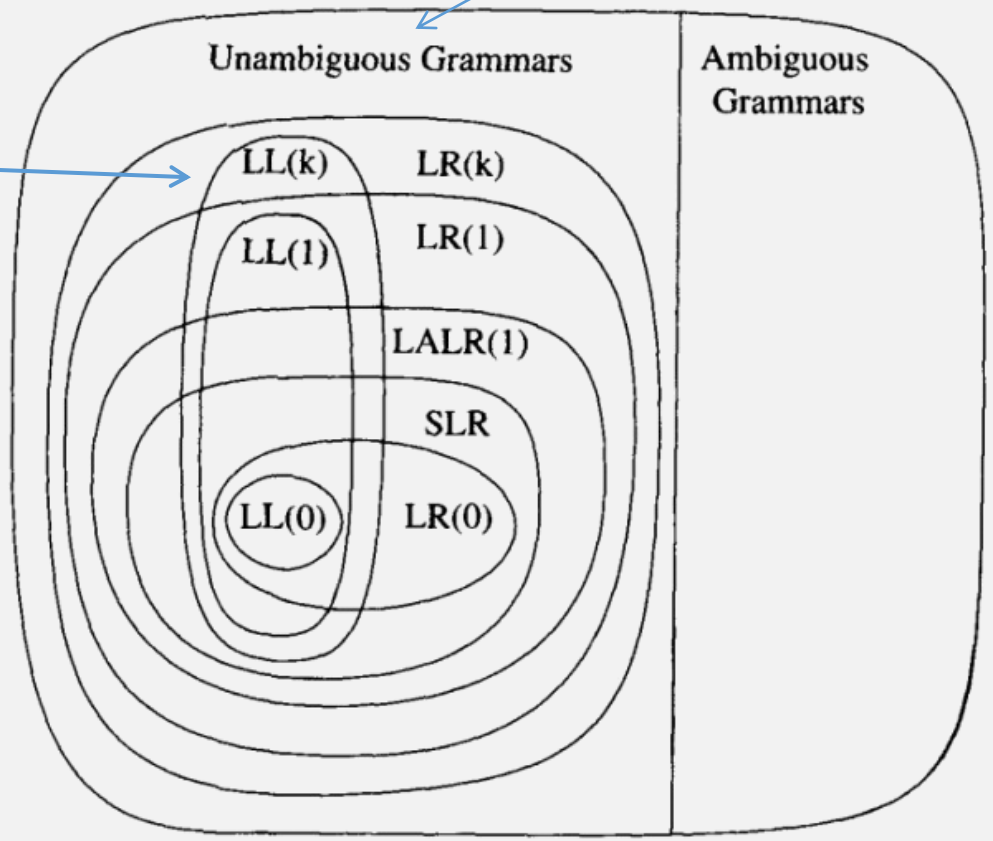
PDAs recognize CFLs, but **DPDAs** only recognize **DCFLs!** (a subset of CFLs)

Subclasses of CFLs

Umambiguous CFLs / PDAs

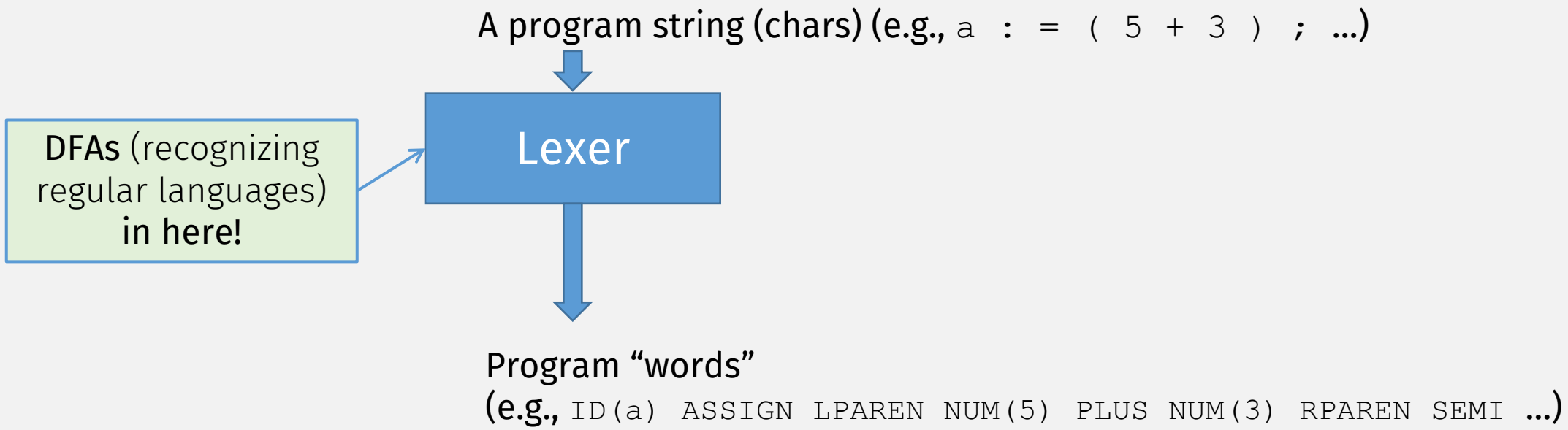
DCFLs

Programming language parsers / compilers are ideally in here



All CFLS

Compiler Stages



A Lexer Implementation

Remember our analogy:
- DFAs are like programs
- All possible DFA tuples is like a programming language

This DFA is a real program!

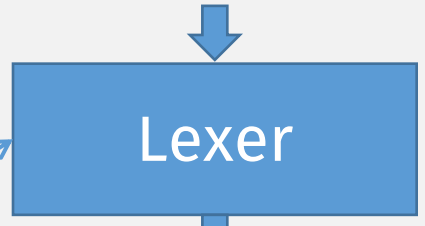
A "lex" tool converts the program:
- from "DFA Lang" ...
- to an **equivalent** one in C !

DFAs (represented as regular expressions)!

```
%{
/* C Declarations: */
#include "tokens.h"    /* definitions of IF, ID, NUM, ... */
#include "errormsg.h"
union {int ival; string sval; double fval;} yylval;
int charPos=1;
#define ADJ  (EM_tokPos=charPos, charPos+=yyleng)
%}
/* Lex Definitions: */
digits   [0-9]+
%%
/* Regular Expressions and Actions: */
if                          {ADJ; return IF;}
[a-z][a-z0-9]*              {ADJ; yylval.sval=String(yytext);
                              return ID;}
{digits}                   {ADJ; yylval.ival=atoi(yytext);
                              return NUM;}
({digits}"."[0-9]*)|([0-9]*"."{digits})      {ADJ;
                              yylval.fval=atof(yytext);
                              return REAL;}
("--"[a-z]*"\n")|(" "|"\n"|"\t")+     {ADJ;}
.                           {ADJ; EM_error("illegal character");}
```

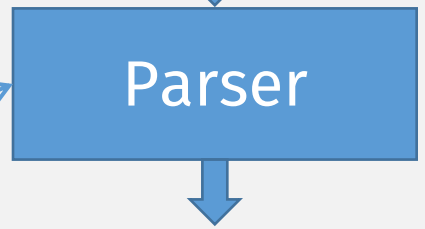
Compiler Stages

A program (chars) (e.g., `a : = ( 5 + 3 ) ; ...`)

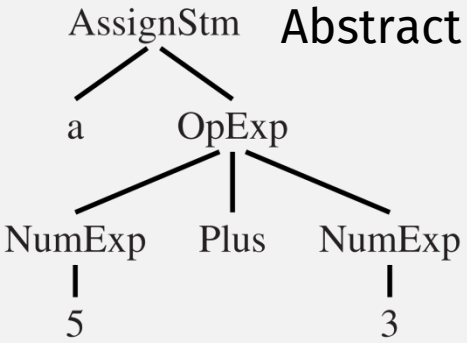


DFAs (recognizing regular languages) **in here!**

Program "words"
(e.g., `ID(a) ASSIGN LPAREN NUM(5) PLUS NUM(3) RPAREN SEMI ...`)



DPDAs (recognizing DCFLs) **in here!**



Abstract Syntax tree (AST), i.e., a **parse tree!**

A Parser Implementation

```
%{
int yylex(void);
void yyerror(char *s) { EM_error(EM_tokPos, "%s", s); }
%}
%token ID WHILE BEGIN END DO IF THEN ELSE SEMI ASSIGN
%start prog
%%

prog: stmlist

stm : ID ASSIGN ID
    | WHILE ID DO stm
    | BEGIN stmlist END
    | IF ID THEN stm
    | IF ID THEN stm ELSE stm

stmlist : stm
        | stmlist SEMI stm
```

Just write the CFG!

Remember our analogy:
CFGs are like **programs**

This CFG is a real program!

A "`yacc`" tool converts the program:
- from "CFG Lang" ...
- to an **equivalent** one in C !

DPDAs are Not Equivalent to PDAs!

Parsing = generating reversed:
- start with string
- end with parse tree

$$R \to S \mid T$$
$$S \to \mathrm{a}S\mathrm{b} \mid \mathrm{ab}$$
$$T \to \mathrm{a}T\mathrm{bb} \mid \mathrm{abb}$$

- **PDA**: can non-deterministically "try all rules" (abandoning failed attempts);
- **DPDA**: must choose one rule at each step!

Should use S rule

$\mathrm{aaabbb} \longmapsto \mathrm{aaSbb}$

aaa

Should use T rule

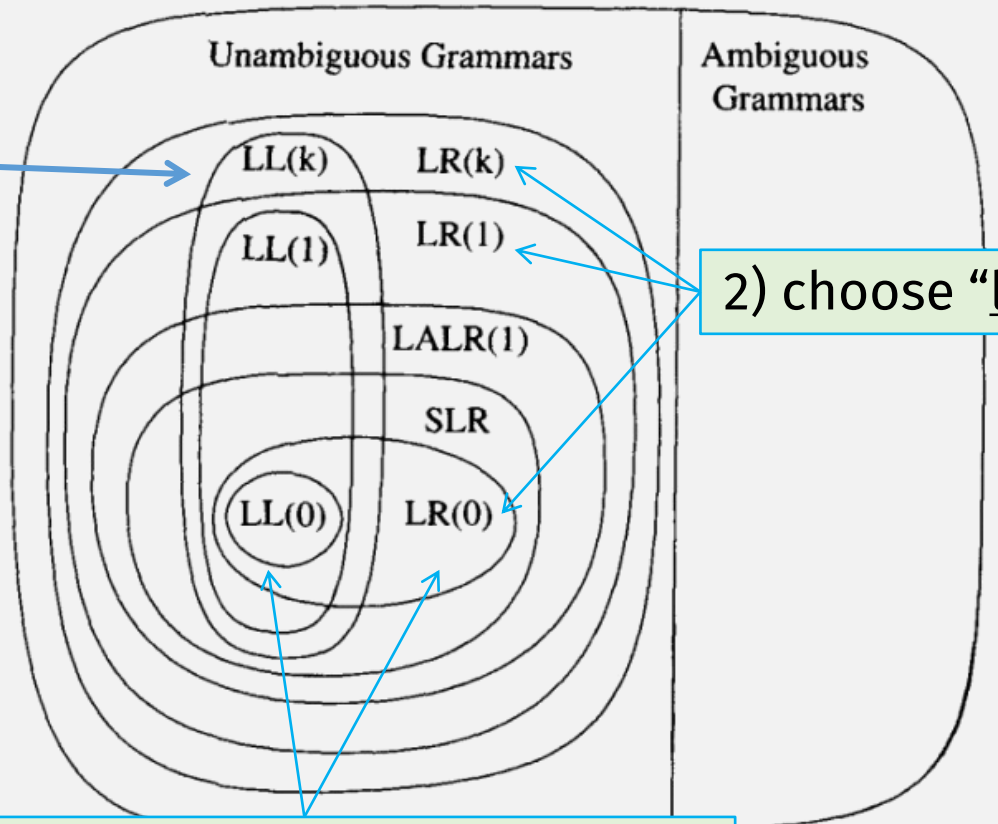
$\mathrm{aaabbbbbb} \longmapsto \mathrm{aaTbbbb}$

When parsing reaches this position, does it know which rule, S or T?

To choose "correct" rule, need to "look ahead" at rest of the input!

PDAs recognize CFLs, but **DPDAs** only recognize **DCFLs!** (a subset of CFLs)

Subclasses of CFLs



DCFLs

Programming language parsers / compilers are ideally in here

2) choose "look ahead" amount

2 parser design decisions:
1) Parse from left, or from right

All CFLS

LL parsing

- **L** = left-to-right
- **L** = leftmost derivation

Game: "You're the Parser": Guess which rule applies?

(and how much did you have to "look ahead"?)

1 $S \rightarrow$ if E then S else S

2 $S \rightarrow$ begin S L

3 $S \rightarrow$ print E

4 $L \rightarrow$ end

5 $L \rightarrow$; S L

6 $E \rightarrow$ num = num

```
if 2 = 3 begin print 1; print 2; end else print 0
```



LL parsing

- **L** = left-to-right
- **L** = leftmost derivation

1 $S \rightarrow \text{if } E \text{ then } S \text{ else } S$

2 $S \rightarrow \text{begin } S\ L$

3 $S \rightarrow \text{print } E$

4 $L \rightarrow \text{end}$

5 $L \rightarrow ;\ S\ L$

6 $E \rightarrow \text{num} = \text{num}$

```
if 2 = 3 begin print 1; print 2; end else print 0
```

LL parsing

- **L** = left-to-right
- **L** = leftmost derivation

1 $S \rightarrow \text{if } E \text{ then } S \text{ else } S$

2 $S \rightarrow \text{begin } S\ L$

3 $S \rightarrow \text{print } E$

4 $L \rightarrow \text{end}$

5 $L \rightarrow ;\ S\ L$

6 $E \rightarrow \text{num} = \text{num}$

```
if 2 = 3 begin print 1; print 2; end else print 0
```



LL parsing

- **L** = left-to-right
- **L** = leftmost derivation

1 $S \rightarrow \text{if } E \text{ then } S \text{ else } S$

2 $S \rightarrow \text{begin } S\ L$

3 $S \rightarrow \text{print } E$

4 $L \rightarrow \text{end}$

5 $L \rightarrow ;\ S\ L$

6 $E \rightarrow \text{num} = \text{num}$

```
if 2 = 3 begin print 1; print 2; end else print 0
```

“Prefix” languages (Scheme/Lisp) **are easily parsed with LL parsers** (zero lookahead)

LR parsing

- **L** = left-to-right
- **R** = rightmost derivation

1 $S \rightarrow S\ ;\ S$

2 $S \rightarrow \text{id}\ :=\ E$

3 $S \rightarrow \text{print}\ (\ L\ )$

4 $E \rightarrow \text{id}$

5 $E \rightarrow \text{num}$

6 $E \rightarrow E\ +\ E$

```
a := 7;
b := c + (d := 5 + 6, d)
```

When parse is here, can't determine whether it's an assign (:=) or addition (+)

Need to save input (lookahead) to some memory, like a **stack**! this is a job for a (D)PDA!

LR parsing

- **L** = left-to-right
- **R** = rightmost derivation

$$S \rightarrow S\ ;\ S \qquad E \rightarrow \text{id}$$
$$S \rightarrow \text{id} := E \qquad E \rightarrow \text{num}$$
$$S \rightarrow \text{print}\ (\ L\ ) \qquad E \rightarrow E + E$$

```
a := 7;
b := c + (d := 5 + 6, d)
```

Stack	*Input*	*Action*
1	a := 7 ; b := c + (d := 5 + 6 , d) $	*shift*

push

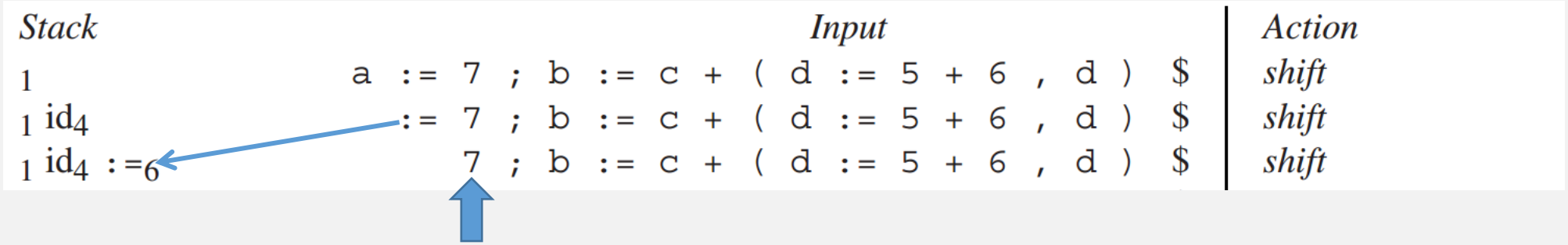
= “push”

State name

LR parsing

- **L** = left-to-right
- **R** = rightmost derivation

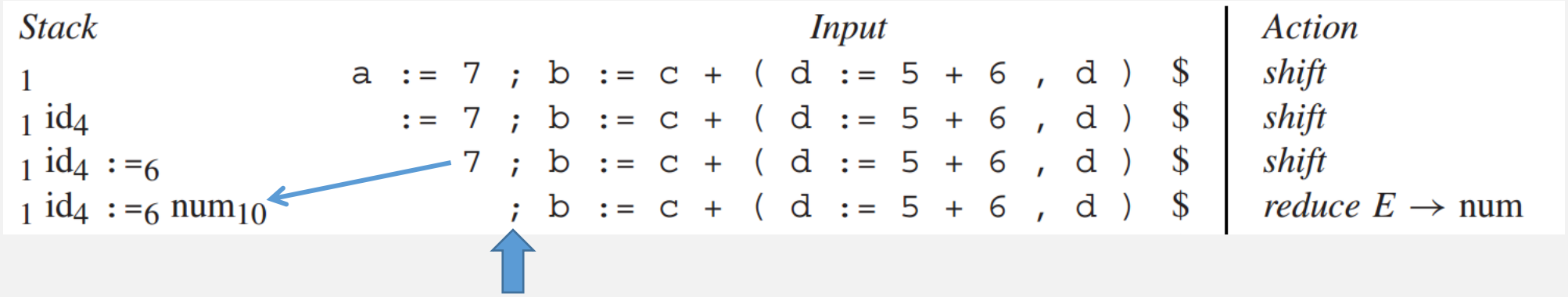
$$S \rightarrow S\ ;\ S \qquad E \rightarrow \text{id}$$
$$S \rightarrow \text{id} := E \qquad E \rightarrow \text{num}$$
$$S \rightarrow \text{print}\ (\ L\ ) \qquad E \rightarrow E + E$$



LR parsing

- **L** = left-to-right
- **R** = rightmost derivation

$$S \rightarrow S\ ;\ S \qquad E \rightarrow \text{id}$$
$$S \rightarrow \text{id} := E \qquad E \rightarrow \text{num}$$
$$S \rightarrow \text{print}\ (\ L\ ) \qquad E \rightarrow E + E$$



LR parsing

- **L** = left-to-right
- **R** = rightmost derivation

1 $S \rightarrow S\ ;\ S$

2 $S \rightarrow \text{id} := E$

3 $S \rightarrow \text{print}\ (\ L\ )$

4 $E \rightarrow \text{id}$

5 $E \rightarrow \text{num}$

6 $E \rightarrow E + E$

Stack	*Input*	*Action*
1	a := 7 ; b := c + (d := 5 + 6 , d) $	*shift*
1 id_4	:= c + (d := 5 + 6 , d) $	*shift*
1 id_4 :=6	:= c + (d := 5 + 6 , d) $	*shift*
1 id_4 :=6 num_{10}	; b := c + (d := 5 + 6 , d) $	*reduce* $E \rightarrow$ num

Can determine (rightmost) rule



LR parsing

- **L** = left-to-right
- **R** = rightmost derivation

1 $S \rightarrow S\ ;\ S$

2 $S \rightarrow \text{id} := E$

3 $S \rightarrow \text{print}\ (\ L\ )$

4 $E \rightarrow \text{id}$

5 $E \rightarrow \text{num}$

6 $E \rightarrow E + E$

Stack	*Input*	*Action*
1	a := 7 ; b := c + (d := 5 + 6 , d) $	*shift*
1 id_4	:= 7 ; b := c + (d := 5 + 6 , d) $	*shift*
1 id_4 :=$_6$	= c + (d := 5 + 6 , d) $	*shift*
1 id_4 :=$_6$ num_{10}	= c + (d := 5 + 6 , d) $	*reduce* $E \rightarrow$ num
1 id_4 :=$_6$ E_{11}	; b := c + (d := 5 + 6 , d) $	*reduce* $S \rightarrow$ id:=E

Can determine (rightmost) rule

LR parsing

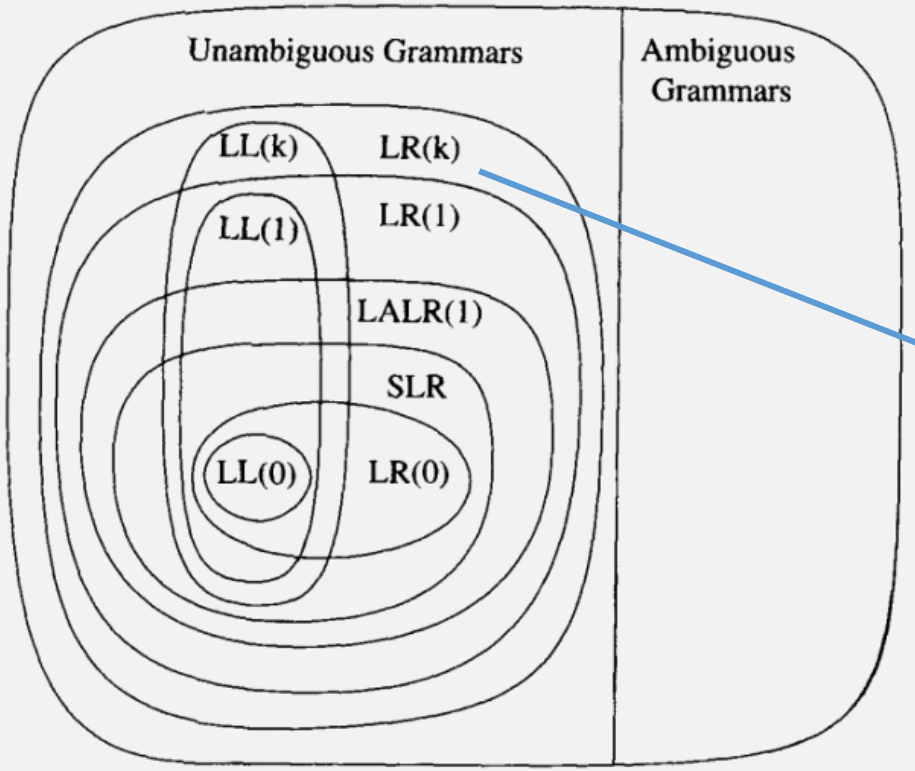
- **L** = left-to-right
- **R** = rightmost derivation

$$S \rightarrow S\ ;\ S \qquad E \rightarrow \text{id}$$
$$S \rightarrow \text{id} := E \qquad E \rightarrow \text{num}$$
$$S \rightarrow \text{print}\ (\ L\ ) \qquad E \rightarrow E + E$$

Stack	*Input*	*Action*
1	a := 7 ; b := c + (d := 5 + 6 , d) $	*shift*
1 id_4	:= 7 ; b := c + (d := 5 + 6 , d) $	*shift*
1 id_4 :=$_6$	7 ; b := c + (d := 5 + 6 , d) $	*shift*
1 id_4 :=$_6$ num_{10}	; b := c + (d := 5 + 6 , d) $	*reduce* $E \rightarrow \text{num}$
1 id_4 :=$_6$ E_{11}	; b := c + (d := 5 + 6 , d) $	*reduce* $S \rightarrow \text{id} := E$
1 S_2	; b := c + (d := 5 + 6 , d) $	*shift*

LR Parsers also called "Shift-Reduce" Parsers

To learn more, take a Compilers Class!



A program (string of chars)

Lexer
(DFAs / NFAs)

Program “words”

Parser
(DPDAs)

Abstract Syntax tree (AST)

???

This phase needs computation that goes beyond CFLs

UMB CS 420

Non-CFLs

Wednesday, March 27, 2024



Announcements

- HW 6
  - Due Monday 4/1 12pm noon

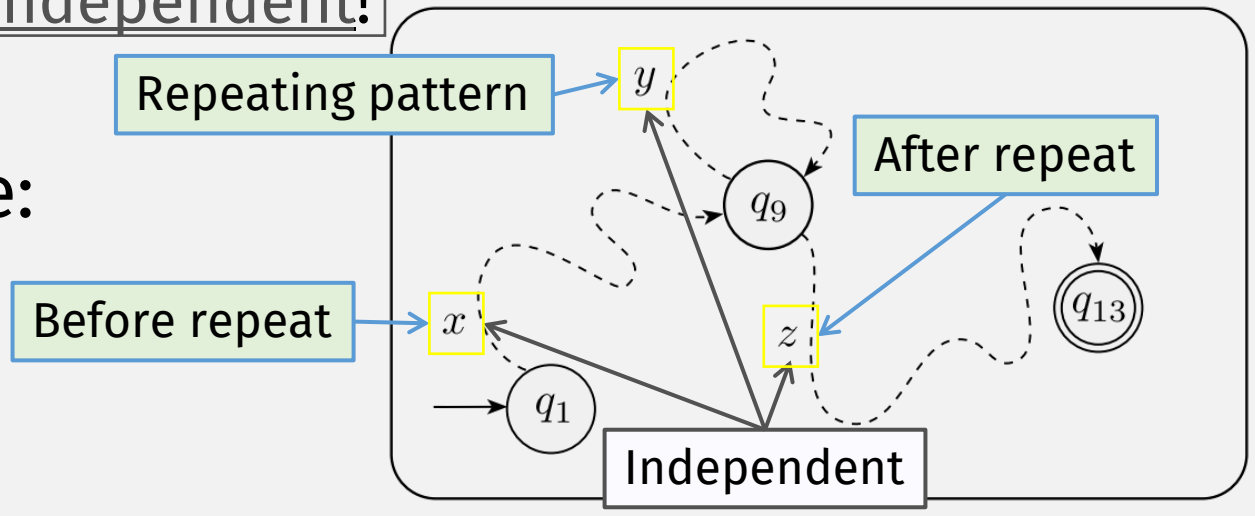


Flashback: Pumping Lemma for Regular Langs

- **Pumping Lemma** describes how strings repeat
- **Regular language** strings repeat using **Kleene star** operation
  - Key: 3 substrings $x\,y\,z$ independent!
- A non-regular language:

$$\{0^n 1^n \mid n \geq 0\}$$

Kleene star can't express this pattern:
2nd part depends on (length of) 1st part



- Q: How do CFLs repeat?

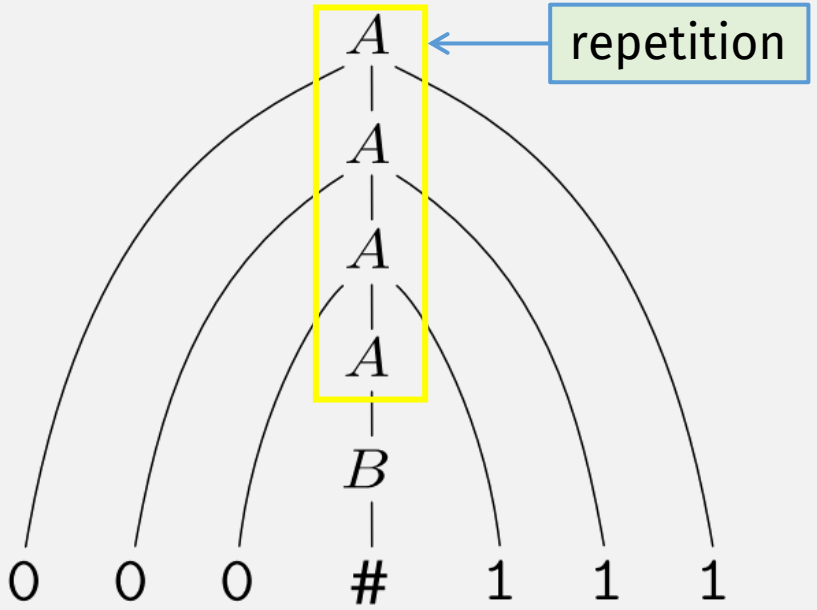
Repetition and Dependency in CFLs

Parts before/after repetition point linked (not independent)

Repetition

$$A \to 0A1$$
$$A \to B$$
$$B \to \#$$

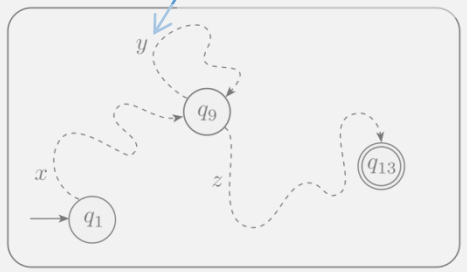
$$\{0^n\#1^n \mid n \geq 0\}$$



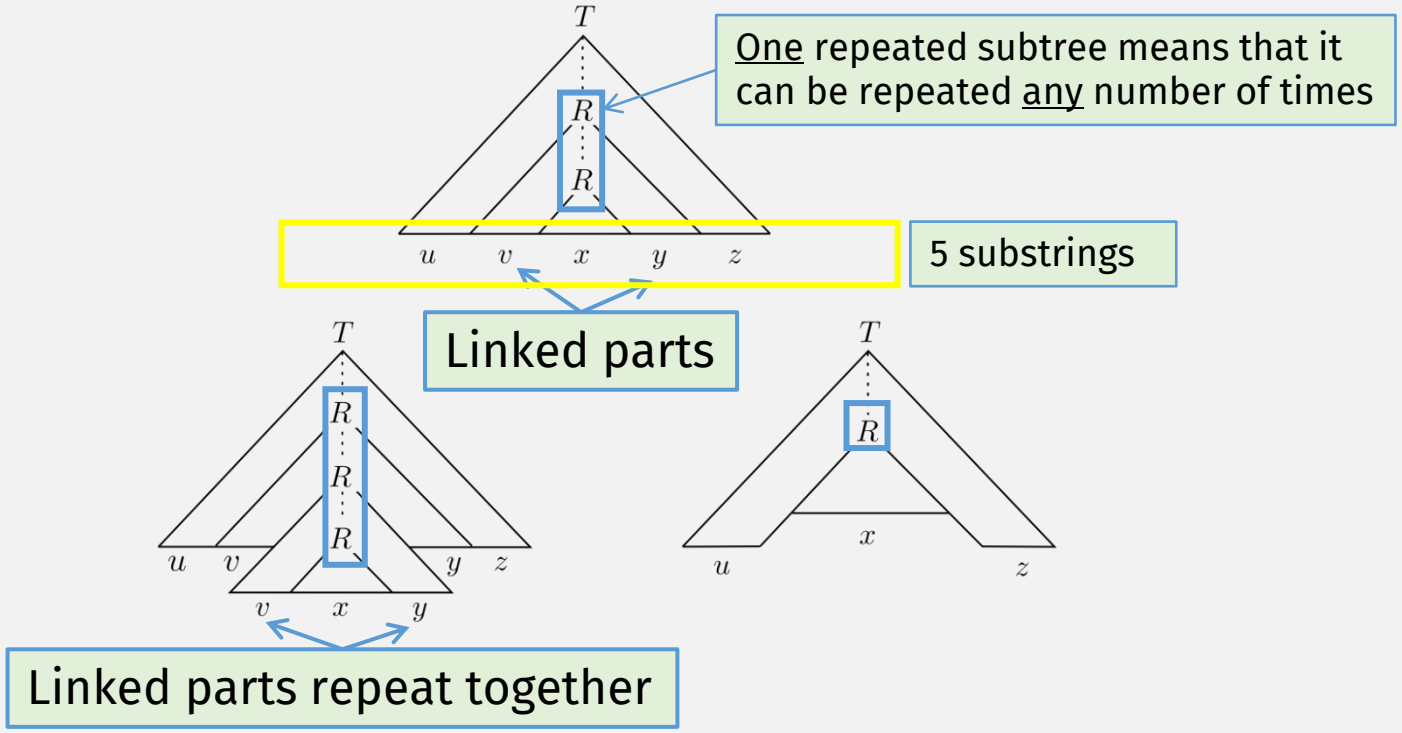
$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$$

How Do Strings in CFLs Repeat?

NFA can take loop transition any number of times, to process repeated y in input



- Strings in regular languages repeat states
- Strings in CFLs repeat subtrees in the parse tree

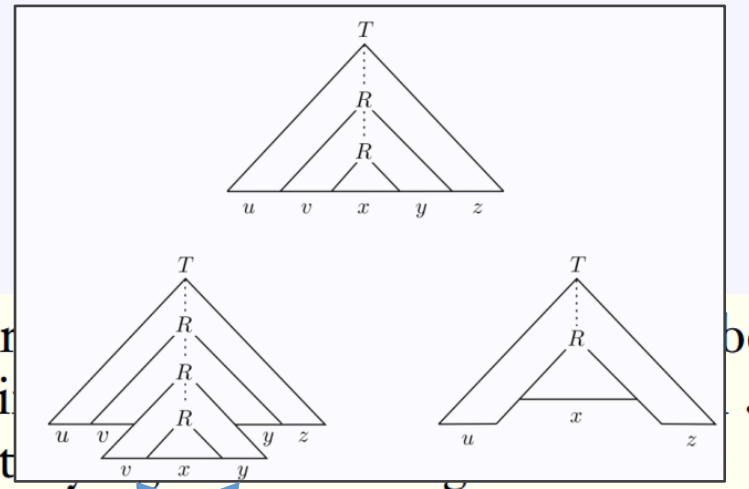


Pumping Lemma for CFLS

Pumping lemma for context-free languages If A is a context-free language, then there is a number p (the pumping length) where, if s is any string in A of length at least p, then s may be divided into five pieces $s = uvxyz$ satisfying the conditions

Two pumpable parts.
But they must be pumped together!

1. for each $i \geq 0$, $uv^ixy^iz \in A$,
2. $|vy| > 0$, and
3. $|vxy| \leq p$.



Two pumpable parts, pumped together

Pumping lemma If A is a regular [illegible] ber p (the pumping length) where if s is any stri [illegible] s may be divided into three pieces, $s = xyz$, sat [illegible]

1. for each $i \geq 0$, $xy^iz \in A$,
2. $|y| > 0$, and
3. $|xy| \leq p$.

One pumpable part

Previously

A Non CFL example

$$\text{language } B = \{\mathtt{a}^n\mathtt{b}^n\mathtt{c}^n \mid n \geq 0\} \text{ is not context free}$$

Intuition

- **Strings in CFLs** can have **two parts** that are **"pumped" together**
- **Language B requires three parts** to be **"pumped" together**
- So **it's not a CFL!**

Proof?

Want to prove: $a^n b^n c^n$ **is not** a CFL

Pumping lemma for context-free languages If A is a context-free language, then there is a number p (the pumping length) where, if s is any string in A of length at least p, then s may be divided into five pieces $s = uvxyz$ satisfying the conditions

1. for each $i \geq 0$, $uv^ixy^iz \in A$,
2. $|vy| > 0$, and
3. $|vxy| \leq p$.

Reminder: CFL Pumping lemma says:
all strings $a^n b^n c^n \geq$ length p are splittable into $uvxyz$ where v and y are pumpable

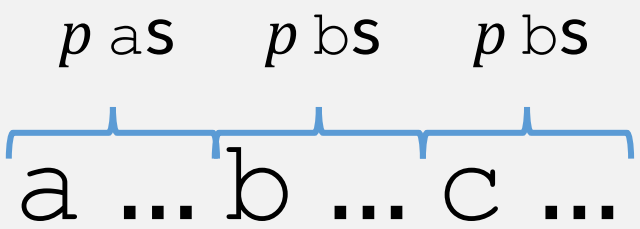
Proof (by contradiction):

Now we must find a contradiction …

- **Assume:** $a^n b^n c^n$ **is a CFL**
  - So it must satisfy the pumping lemma for CFLs
  - I.e., all strings ≥ length p are pumpable
- **Counterexample** = $a^p b^p c^p$

Contradiction if:
- A string in the language ☑
- ≥ length p ☑
- Is **not splittable** into $uvxyz$ where v and y are pumpable

???



Want to prove: $a^n b^n c^n$ **is not** a CFL

Pumping lemma for context-free languages If A is a context-free language, then there is a number p (the pumping length) where, if s is any string in A of length at least p, then s may be divided into five pieces $s = uvxyz$ satisfying the conditions

1. for each $i \geq 0$, $uv^i xy^i z \in A$,
2. $|vy| > 0$, and
3. $|vxy| \leq p$.

Possible Splits

Proof (by contradiction):

- Assume: $a^n b^n c^n$ **is** a CFL
  - So it must satisfy the pumping lemma for CFLs
  - I.e., all strings ≥ length p are pumpable
- Counterexample = $a^p b^p c^p$

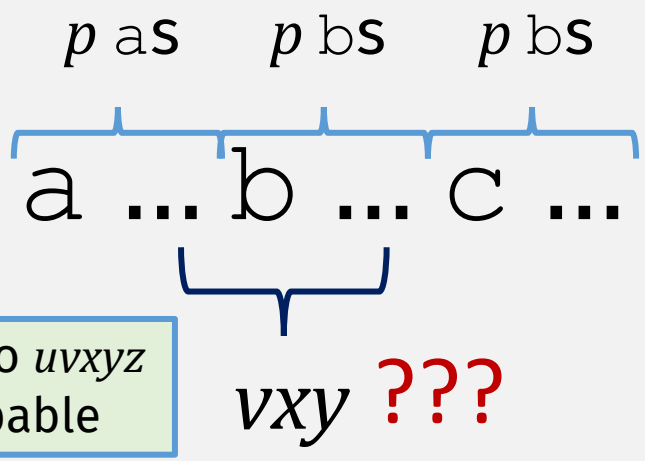
contradiction

Contradiction if:
- A string in the language
- ≥ length p
- Is **not splittable** into $uvxyz$ where v and y are pumpable

- Possible Splits (using condition # 3: $|vxy| \leq p$)
  - ✘ vxy is all as
  - ✘ vxy is all bs
  - ✘ vxy is all cs
  - ✘ vxy has as and bs
  - ✘ vxy has bs and cs
  - (vxy cannot have as, bs, and cs)

Not pumpable

$a^p b^p c^p$ **cannot be split** into $uvxyz$ where v and y are pumpable

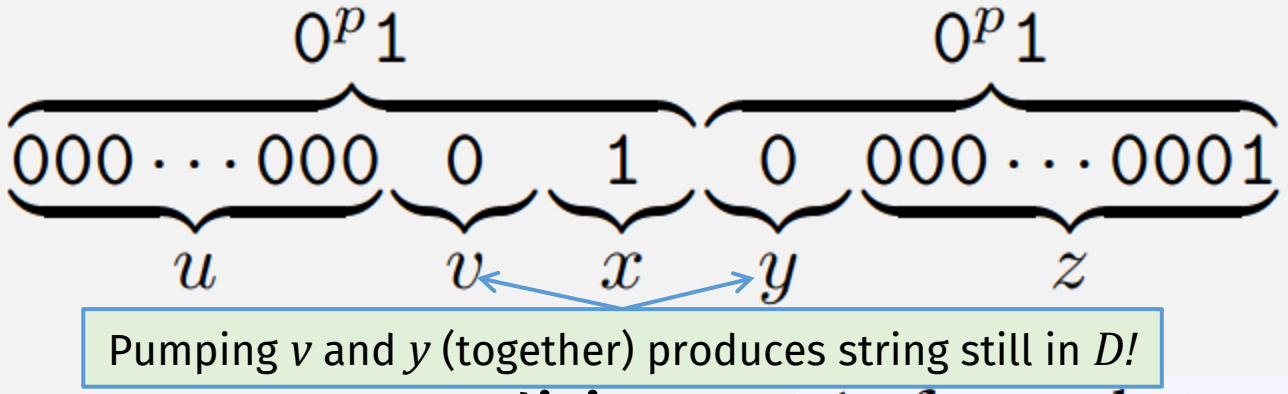


So $a^n b^n c^n$ **is not a CFL**
(justification:
contrapositive of CFL pumping lemma)

Another Non-CFL $D = \{ww | \ w \in \{0,1\}^*\}$

Be careful when choosing **counterexample** s: 0^p10^p1

This s can be pumped according to **CFL pumping lemma**:



- CFL Pumping Lemma conditions:
  - ☑ **1.** for each $i \geq 0$, $uv^ixy^iz \in A$,
  - ☑ **2.** $|vy| > 0$, and
  - ☑ **3.** $|vxy| \leq p$.

So **this attempt** to **prove** that the **language** is **not** a **CFL failed**.
(It doesn't prove that the language is a CFL!)

Another Non-CFL $D = \{ww | \ w \in \{0,1\}^*\}$

- Need another counterexample string s:

If vyx is contained in first or second half, then any pumping will break the match ✗

$$0^p 1^p 0^p 1^p$$

So vyx must straddle the middle

But any pumping still breaks the match because order is wrong ✗

- CFL Pumping Lemma conditions:

1. for each $i \geq 0$, $uv^i xy^i z \in A$,
2. $|vy| > 0$, and
3. $|vxy| \leq p$.

Now we have proven that this language is **not a CFL!**

A Practical Non-CFL

- **XML**
  - ELEMENT → <TAG>CONTENT</TAG>
  - Where TAG is any string

- XML also looks like this non-CFL: $D = \{ww | \ w \in \{0,1\}^*\}$

- This means XML is not context-free!
  - Note: HTML *is* context-free because …
  - … there are only a finite number of tags,
  - so they can be embedded into a finite number of rules.

In practice:
- XML is parsed as a CFL, with a CFG
- Then matching tags checked in a 2nd pass with a more powerful machine …

Next: A More Powerful Machine …

M_1 accepts its input if it is in language: $B = \{w\#w \mid w \in \{0,1\}^*\}$

M_1 = "On input string w:

1. Zig-zag across the tape to corresponding positions on either side of the # symbol to check whether these positions contain the same symbol. If they do not, or if no # is found, *reject*. Cross off symbols as they are checked to keep track of which symbols correspond.

Infinite memory (initial contents are the input string)

Can move to, and read/write from arbitrary memory locations!